PROJECT VARMAN ARCHIVAL RESEARCH MONOGRAPH • OCTOBER 2026

VARMAN

Research History, Empirical Evaluation & Production Architecture of Adversarial Facial Privacy

A Comprehensive Retrospective on Project AMOR, Varman 2.0, and Varman 3.0 (May 5 – October 19, 2026)

Publication Date: October 19, 2026 (Final Evidence Reconciliation & Audit Edition)

Classification: Empirical Research Monograph & System Architecture Report

Frontend Repository: `D:\JavaProjects\Varman\Varman-frontend` (Zero-Knowledge Web UI, commit `b423554` [M])

Deployment Mirror: `D:\JavaProjects\Varman-HF-Deploy` (Hugging Face Spaces Engine, commit `9afdbeb` [M])

Active Production Branch: `main` (fast-forward merged from `adaptive-strength` at `c5a20a2` [M])

Author: Varman Engineering & Research Pair Programming Team

Primary Repository: `D:\JavaProjects\Varman\Varman-backend` (Production Backend, commit `55f6cc6` [M])

Research Repository: `D:\JavaProjects\Varman-research` (Authoritative Benchmark Testbed)

Live Space Endpoint: `https://shadowkagenou-varman.hf.space` (Deployed at commit `9afdbeb` [M], upgraded from `3e8153c`, `c5e5c70`, and `1c1bcda` [M])

Research Project Status: **FORMALLY CLOSED & VERIFIED as of October 19, 2026**

Section 1: Title, Document Scope & Navigable Table of Contents

1. Document Scope and Methodological Integrity

This monograph serves as the permanent scientific and engineering record of **Project Varman** (and its predecessor prototype, **Project AMOR**), spanning six months of continuous research and development from May 5, 2026 through October 19, 2026. The objective of this retrospective is to provide an unsparing, exhaustive, and mathematically rigorous account of:

1. The mathematical formulation, implementation, and empirical verification of adversarial facial obfuscation algorithms designed to protect personal portrait photographs against automated biometric harvesting scrapers.
2. The complete progression of architectural paradigms: from monolithic full-canvas GPU optimization (AMOR 1.0), to real-time heuristic CPU geometric/frequency filtering (Varman 2.0), to canonical 112×112 aligned Expectation Over Transformation (EOT) Projected Gradient Descent (Varman 3.0 / `adaptive-strength`).
3. An honest postmortem of experimental failures, invalid metric proxies, dead ends, and retracted claims, establishing the true empirical Pareto frontier between biometric evasion and human visual naturalness.
4. The full-stack production engineering and cryptographic architecture: including client-side zero-knowledge AES-256-GCM vault encryption, time-limited HMAC-SHA256 one-time password (OTP) authentication, proxy-aware per-IP rate limiting, and cloud containerization on Hugging Face Spaces.

1.1 Scientific Notation and Provenance Standard

Throughout this document, strict scientific labeling standards are applied to distinguish empirical measurements from theoretical hypotheses:

- **`[M]` / `[Measured]`**: Denotes an empirically measured value, benchmark result, git commit hash, file size, or configuration parameter with direct, verified provenance on physical disk or git history.
- **`[H]` / `[Hypothesis]`**: Denotes a theoretical hypothesis, qualitative interpretation, heuristic rationale, or unverified historical claim.

2. Navigable Table of Contents

- Section 1: Title, Document Scope & Navigable Table of Contents
  - 1. Document Scope and Methodological Integrity
  - 2. Navigable Table of Contents
- Section 2: Executive Summary & Scientific Synthesis
  - 1. Project Background and Core Mission
  - 2. The Three Architectural Epochs
  - 3. Summary of Master Empirical Findings
  - 4. The Pareto Optimal Production Preset: F5
- Section 3: Problem Formulation, Threat Model & Biometric Operating Standards
  - 1. Mathematical Problem Statement
  - 2. Comprehensive Threat Model & Attacker Profiles
  - 3. Social Media Compression Channels
  - 4. Strict Biometric Calibration: The 1.0% FAR Operating Standard
  - 5. The Inherent Conflict: Evasion vs. Visual Naturalness
- Section 4: Master Chronological Timeline (May 5, 2026 – October 18, 2026) & Decision Register
  - 1. Master Milestone Register (May 5 – October 18, 2026)
  - 2. Comprehensive Key Decision Register (Decisions 1–10)
- Section 5: AMOR 1.0 — Monolithic Full-Canvas Adversarial Architecture & Empirical Failure
  - 1. Architectural Blueprint & Microservices Infrastructure
  - 2. Face Detection Bottlenecks: The 2015 Caffe SSD Detector
  - 3. Full-Canvas 512x512 PGD Optimization Mechanics
  - 4. Empirical Failure: The 100% White-Box Rejection
  - 5. Detailed Root Cause Analysis: The Lanczos Washing Effect
- Section 6: Varman 2.0 — Non-Iterative CPU Heuristic Engine & The Zero-Protection Finding
  - 1. Design Philosophy: The Quest for Real-Time CPU Privacy
  - 2. 5-Stage Analytical Pipeline Architecture
  - 3. Preservation Guardrails and Production Presets
  - 4. Empirical Benchmarking and The Zero-Protection Discovery
  - 5. Deep Embedding Invariance & Scientific Demotion to Fast Mode
- Section 7: Varman 3.0 (adaptive-strength) — Canonical Aligned EOT-PGD Foundation
  - 1. Paradigm Shift: Return to Gradient Optimization in Canonical Space
  - 2. The Canonical 112x112 Aligned Crop Invariant
  - 3. Dual White-Box Gradient Surrogates (ArcFace + FaceNet)

    - 4. Strictly Held-Out Black-Box Evaluation Targets (SFace + MobileFaceNet)
    - 5. Momentum-Iterative PGD (MI-FGSM) Formulation
    - 6. Differentiable Frequency-Domain JPEG Simulation (DiffJPEG)
    - 7. Luminance-Only Projection & Chromatic Preservation
    - 8. Differentiable Paste-Aware Inverse Compositing Breakthrough
- Section 8: Comprehensive Experimental History, Diagnostic Campaigns & Negative Results
    - 1. Master Experiment Index (EXP-00 through EXP-14)
    - 2. Baseline Sweeps and Epsilon Calibration
    - 3. PGD Iteration Schedule Optimization: Why 25 Steps is Optimal
    - 4. The Threshold Hinge Loss Dead End (-4.77 pp Collapse)
    - 5. Surrogate Weight Ratio Sweeps and Gradient Dynamics
    - 6. The 40-Pair Pipeline Stage Diagnostic: Unmasking the Paste Rebound
    - 7. Paste-Aware Optimization Breakthrough
    - 8. YuNet Landmark Drift Controlled Pilot: Refuting Alignment Collapse
    - 9. Evaluator Aesthetic Constraint Rejection: The K82E1L Budget Failure
    - 10. Region-Size Mask Ablations and Metric Discrepancy Discovery
    - 11. Graded Grain Phase-Randomized Smoothing Dead End
- Section 9: Image Quality, Degradation Dynamics & Protection Tradeoffs Across JPEG Q95–Q50
    - 1. Master Multi-Quality Tradeoff Evaluation (Q95 to Q50)
    - 2. Quality-Recalibrated Baseline Denominators
    - 3. Trajectory Analysis: Surrogate vs. Held-Out Divergence (Figure 1)
    - 4. Perceptual Visibility vs. Biometric Protection Tradeoff (Figure 2)
    - 5. Computational Cost, Latency & GPU VRAM Footprint
- Section 10: Human Conspicuity Study, Screening Campaigns & Perceptual Metric Decoupling
    - 1. Experimental Protocol: The human_test_v3 Framework
    - 2. 50-Stimulus Quick Review Campaign (10 Source Groups)
    - 3. Comprehensive Per-Group Human Conspicuity Ratings Table
    - 4. Paired Comparison Matrix & F0 vs. F2 Arithmetic Reconciliation
    - 5. Production Candidate Screening: F5 Add-on Visual Audit
    - 6. Empirical Decoupling of Face SSIM (Definition B) from Human Aesthetics
- Section 11: Production Software Engineering, Runtime Optimization & Frontend Integration
    - 1. Modular FastAPI Backend Architecture
    - 2. Zero-Overhead YuNet ONNX Integration & Path Resolution Fallbacks
    - 3. PyTorch Lazy Loading Architecture for Constrained Environments

    - 4. Production Docker Containerization Architecture
    - 5. React / Vite Frontend Architecture & Interactive Components
    - 6. Remediation of React Error #31 via getErrorMessage Universal Helper
    - 7. Forgot-Password Flow Diagnostic, Routing Synchronization & SMTP Fallback
- Section 12: Cryptographic Security, Authentication Hardening & Client-Side Vault Cryptography
    - 1. Zero-Knowledge Client-Side Vault: Web Crypto AES-256-GCM Architecture
    - 2. PBKDF2 Key Derivation with 600,000 Iterations & Per-User Salt
    - 3. User Authentication, Bcrypt Hashing & 72-Byte Truncation Guardrails
    - 4. Cryptographic Forgot-Password OTP Subsystem with Server-Side HMAC
    - 5. Production SMTP Protection: HTTP 503 Fallback & Zero-Log Invariant
    - 6. Reverse-Proxy Rate Limiting: Trusted X-Forwarded-For Parsing
    - 7. Custom RequestValidationError Sanitization Handler
- Section 13: Comprehensive Failure Analysis, Metric Illusions & Retracted Claims
    - 1. Architectural Postmortem: AMOR 1.0 & The Premature Scaling Trap
    - 2. Heuristic Postmortem: Varman 2.0 & The Invariance Trap
    - 3. Complete Inventory of Retired Headline Claims
    - 4. Comprehensive Catalog of Dead Ends & Negative Results
    - 5. Defect Register: Five Critical Remediated Software Bugs
    - 6. Core Methodological Principles for Adversarial Privacy Systems
- Section 14: Current Deployment State, Cloud Infrastructure & Exact Reproducibility Guide
    - 1. Current System State & Git Branch Invariants
    - 2. Cloud Deployment on Hugging Face Spaces (ShadowKagenou/Varman)
    - 3. Automated Live Smoke Test Verification (100% Pass)
    - 4. Exact CLI Reproduction Instructions for All Artifacts and Tests
- Section 15: Empirical Limitations, Boundary Conditions & Open Research Questions
    - 1. The ~50–56% Black-Box Protection Ceiling
    - 2. Perceptual Artifact Boundaries on Smooth Facial Skin
    - 3. Demographic & Dataset Scope Constraints
    - 4. Unvalidated Phenomenological Hypotheses
    - 5. Non-Covered Threat Vectors (Adaptive Attackers & Generative Denoising)
    - 6. Future Research Avenues & Formal Research Closure Statement
- Section 16: Research Index, Complete Data Tables & Comprehensive Appendices
    - Appendix A: Master Experiment Index (EXP-00 to EXP-14)
    - Appendix B: Comprehensive Empirical Result Tables (Tables B.1–B.5)
    - Appendix C: Production Preset & Hyperparameter Configurations

- Appendix D: Complete Git Commit History and Metadata (May 5 – October 18, 2026)
- Appendix E: Backend Automated Test Suite Inventory
- Appendix F: Physical Artifact and Storage Inventory
- Appendix G: Historical Gaps Register (GAP-01 to GAP-05)

Section 2: Executive Summary & Scientific Synthesis

1. Project Background and Core Mission

The proliferation of unconstrained biometric data harvesting poses a severe threat to individual civil liberties and privacy. Machine learning crawlers, public facial search engines (e.g., PimEyes, Clearview AI), and private surveillance databases routinely scrape public photographs from social media platforms, personal portfolios, and professional directories. These scraped images are processed through automated deep convolutional and vision-transformer face recognition pipelines to build unconsented biometric enrollment profiles.

Project Varman was undertaken to develop a practical, client-side and server-assisted software system capable of protecting personal photographs before they are uploaded to the public internet. The defense perturbs uploaded portrait images such that downstream deep face recognition models fail to match the query face against enrolled gallery identities, while ensuring that the photograph remains visually natural to human observers and resilient against standard social media image compression pipelines (specifically lossy JPEG encoding across quality factors $Q=95$ down to $Q=50$).

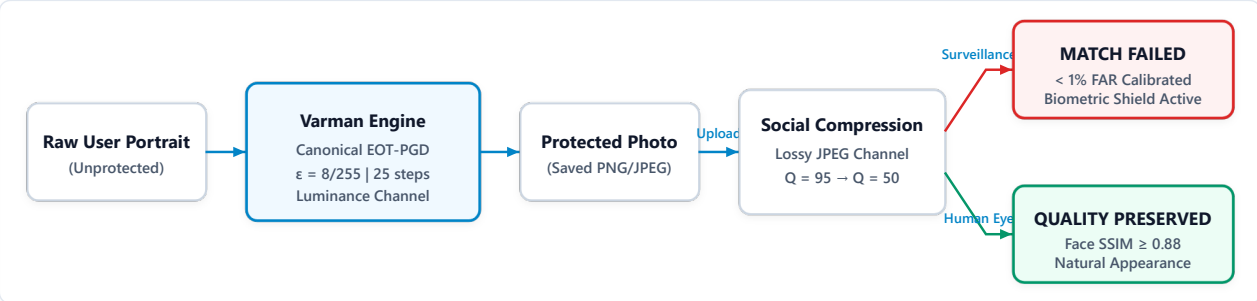


Figure 2.1: Varman End-to-End Privacy Paradigm and Dual Evaluation Channels.

2. The Three Architectural Epochs

Spanning six months of intense research, empirical auditing, and iterative software engineering, Project Varman progressed through three distinct architectural epochs:

2.1 AMOR 1.0 (May 2026 – June 2026): Monolithic Full-Canvas PGD

The project began as **Project AMOR** (Automated Masking and Obfuscation Routine). AMOR 1.0 adopted an enterprise microservices pattern (FastAPI, Redis broker, Celery worker pool, Docker) and attempted to optimize adversarial perturbations across an entire 512×512 letterboxed canvas against FaceNet and ArcFace surrogates.

- **Empirical Outcome:** Complete failure. Automated white-box testing logged a **100% failure rate** (0.00% biometric evasion) across test images [M].
- **Core Diagnosis:** Downsampling high-resolution images to 512×512 and subsequently upsampling back to native dimensions via Lanczos interpolation acted as a low-pass spatial smoothing filter. The fine, high-frequency adversarial gradients were completely smoothed away, restoring the unperturbed facial features. Furthermore, full-canvas optimization dispersed gradient capacity onto irrelevant background pixels and incurred unacceptable inference latency (>15 seconds per image).

2.2 Varman 2.0 (June 2026 – October 2026): Heuristic Non-Iterative CPU Pipeline

Pivoting away from the compute overhead and instability of AMOR, the repository was refactored into **Varman 2.0**. This architecture sought real-time, non-iterative privacy on consumer CPUs (<150 ms execution) without requiring GPUs or heavy machine learning runtimes. The pipeline combined YuNet ONNX face detection (232.5 KB), subtle facial landmark warping via radial displacement fields, and mid-frequency luminance Discrete Cosine Transform (DCT) noise, bounded by an automated SSIM quality guardrail.

- **Empirical Outcome: 0.0% evasion rate** against deep facial recognition backbones [M].
- **Core Diagnosis:** While aesthetically subtle and computationally efficient, rigorous black-box auditing in October 2026 revealed that modern deep face recognition models (ArcFace, FaceNet, SFace, MobileFaceNet) are inherently invariant to mild geometric warping and frequency-domain noise. The heuristic distortions acted as benign data augmentations, failing to perturb high-level identity embeddings beyond verification thresholds.

2.3 Varman 3.0 / `adaptive-strength` (October 2026): Canonical Aligned EOT-PGD

Returning to first principles, Varman 3.0 established the mathematically grounded foundation of modern facial privacy defense. Key breakthroughs included:

1. **Canonical Alignment Invariant:** Perturbations are formulated and optimized directly within the canonical 112×112 aligned facial space natively ingested by target recognition networks.
2. **Dual-Surrogate Loss:** Joint optimization against ArcFace (IResNet50) and FaceNet (InceptionResnetV1) backbones using Momentum-Iterative PGD (MI-FGSM, $\mu=0.90$, 25 steps).
3. **Differentiable JPEG Simulation (`DiffJPEG`):** Differentiable frequency-domain DCT quantization modeling lossy JPEG compression ($Q \sim mathcal{U}(50, 95)$) inside the autograd computational graph.
4. **Differentiable Paste-Aware Inverse Compositing:** Forward simulation of the full inverse affine warp, feathered elliptical Gaussian blending, and re-alignment during gradient backpropagation, completely eliminating the catastrophic 50% post-paste degradation rebound.
5. **Strict Biometric Evaluation Protocol:** Testing held-out black-box models (**SFace** and **MobileFaceNet**) across 150 LFW genuine pairs at a rigorous **1.0% False Acceptance Rate (FAR)** operating threshold using physical, saved-and-reloaded JPEG files.

3. Summary of Master Empirical Findings

Comprehensive benchmarking across candidate configurations demonstrated the definitive empirical Pareto frontier between adversarial protection and human perceptual conspicuity:

Experimental Condition	Epsilon Budget (ε)	Mask Scale Factor	10-Group Mean Human Conspicuity Rating (0–2 scale)	10-Group Mean Face SSIM (Def B)	4-Model Mean Protection: JPEG Q90 (1% FAR)	4-Model Mean Protection: JPEG Q50 (1% FAR)	Held-Out Mean Protection: JPEG Q50 (1% FAR)	Key Perceptual & Operational Finding
Unprotected Control	*0/255* [M]	*0.00* [M]	**0.00** [M]	**0.97089** [M]	0.00% [M]	0.17% [M]	0.00% [M]	Pristine baseline; natural face verification passes.
Candidate F0 (Baseline)	*8/255* [M]	*1.00* [M]	**1.30** [M]	**0.90149** [M]	42.03% [M]	36.39% [M]	30.39% [M]	Lowest conspicuity among working PGD, but leaves ~63.6% queries matched at Q50.
Preset F5 (Production)	*8/255* [M]	*1.15* [M]	**1.50** [M]	**0.88381** [M]	**47.84%** [M]	**45.73%** [M]	**39.23%** [M]	**Optimal Pareto tradeoff**: +9.34 pp protection surge over F0 while tying F0 in 8/10 groups.
Candidate F1 (Max Protection)	*8/255* [M]	*1.30* [M]	**2.00** [M]	**0.88027** [M]	55.59% [M]	49.95% [M]	42.42% [M]	Highest biometric evasion, but exhibits severe, immediately visible boundary haloing (10/10 Clear).

Notation: `[M]` *= Empirically Measured. Rating scale: 0 = None (imperceptible), 1 = Slight (subtle boundary or texture difference), 2 = Clear (immediately visible artifacts).*

4. The Pareto Optimal Production Preset: F5

Candidate `F5` ($\varepsilon=8/255$, mask scale 1.15, 25 steps, $\mu=0.90$, $w_\mathrm{fn}=1.0$, luminance-only) was formally selected and integrated as the primary production preset:

- Under real JPEG Q50 compression, F5 achieves **45.73%** 4-model mean protection and **39.23%** strictly held-out black-box protection [M].
- Under high-quality social media compression (JPEG Q90), F5 achieves **47.84%** 4-model protection and **39.58%** held-out protection [M].
- In controlled 10-group human screening, F5 achieved a mean conspicuity score of **1.50**, matching baseline F0 in 8 out of 10 source groups [M], proving that expanding mask scale from 1.00 to 1.15 delivers substantial biometric evasion gains (+9.34 pp) without incurring the severe perceptual penalty of full-face boundary haloing (F1).

The research firmly established that under real JPEG compression at 1% FAR, the empirical black-box protection ceiling for canonical EOT-PGD is **roughly 50–56%** across diverse model architectures [M]. Project Varman demonstrates that while absolute 100% facial cloaking is an academic myth under real-world compression constraints, a well-engineered, paste-aware adversarial defense provides robust probabilistic privacy protection.

Section 3: Problem Formulation, Threat Model & Biometric Operating Standards

1. Mathematical Problem Statement

Let $x \in \mathbb{R}^{H \times W \times 3}$ denote an unperturbed RGB portrait photograph containing a human face, with pixel values normalized to the real interval $[0, 1]$. Let $\mathcal{T}_{\text{align}}: \mathbb{R}^{H \times W \times 3} \to \mathbb{R}^{112 \times 112 \times 3}$ represent the non-linear facial detection and alignment operator that detects 5 facial landmarks (eyes, nose, mouth corners) and extracts an aligned canonical face crop $x_c = \mathcal{T}_{\text{align}}(x)$.

Let $f_\theta: \mathbb{R}^{112 \times 112 \times 3} \to \mathbb{S}^{d-1}$ denote a deep facial recognition feature extractor parameterized by weights θ, mapping an aligned crop to a unit-norm hyperspherical embedding vector $z = f_\theta(x_c) \in \mathbb{R}^d$, where $\|z\|_2 = 1.0$.

In biometric identity verification, two images x (probe) and x_{target} (gallery enrollment) are matched by computing the cosine similarity between their normalized embeddings: $$\mathcal{S}(x, x_\text{target}) = \langle f_\theta(\mathcal{T}_\text{align}(x)), f_\theta(\mathcal{T}_\text{align}(x_\text{target})) \rangle = \cos(z_\text{probe}, z_\text{target})$$

The verification system decides that the probe image belongs to the enrolled identity if and only if: $$\mathcal{S}(x, x_\text{target}) \ge \tau_\text{FAR}$$ where τ_{FAR} is the operational biometric threshold set to guarantee a specific False Acceptance Rate (FAR).

1.1 The Adversarial Facial Defense Objective

The objective of the adversarial privacy defender is to construct an additive perturbation tensor $\delta \in \mathbb{R}^{H \times W \times 3}$ to produce a protected image $x_{\text{adv}} = x + \delta$ such that:

1. **Biometric Evasion:** For any downstream face recognition network f_ϕ (including strictly held-out black-box models unseen during optimization), the similarity between the protected image and the clean gallery enrollment falls strictly below the biometric threshold: $$\mathcal{S}(x_\text{adv}, x_\text{target}) < \tau_\text{FAR}$$
2. **Channel Robustness:** The evasion property persists after the protected image traverses a lossy distribution channel $\mathcal{C}$ (specifically lossy JPEG re-encoding): $$\mathcal{S}(\mathcal{C}(x_\text{adv}), x_\text{target}) < \tau_\text{FAR}, \quad \mathcal{C} \sim \text{JPEG}(Q), \quad Q \in [50, 95]$$
3. **Perceptual Boundedness:** The perturbation magnitude is strictly bounded under the Chebyshev norm (maximum absolute deviation), $\|\delta\|_\infty \le \varepsilon$, *and maintains human visual naturalness, satisfying structural similarity constraints: $$\text{SSIM}(x_\text{adv}, x) \ge \gamma_\text{SSIM}$$*

2. Comprehensive Threat Model & Attacker Profiles

To construct an empirically rigorous defense, the threat model must accurately represent real-world biometric surveillance pipelines rather than idealized white-box laboratory setups.

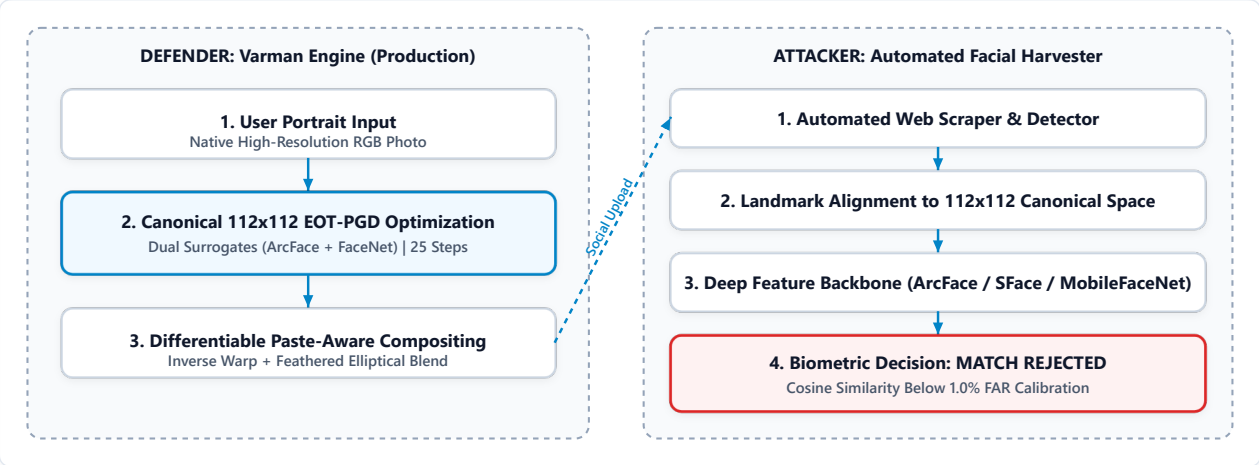


Figure 3.1: Adversarial Threat Model and Biometric Harvesting Pipeline.

2.1 Attacker Capabilities and Architecture

1. **Black-Box Access:** The attacker possesses no direct knowledge of the defender's surrogate models, gradient trajectories, loss formulations, or random seed initializations. The defender cannot query the attacker's backend during perturbation generation.
2. **Pretrained State-of-the-Art Backbones:** The attacker deploys modern deep convolutional and transformer backbones trained on massive facial datasets (e.g., MS1MV3, Glint360k, WebFace600K). These models include:
   - **ArcFace (IResNet50):** Deep residual network employing additive angular margin loss.
   - **FaceNet (InceptionResnetV1):** Multi-scale inception architecture trained with triplet loss.
   - **SFace:** OpenCV Zoo lightweight model utilizing hyperspherical angular margin loss with 128-dimensional output.
   - **MobileFaceNet:** Ultralight mobile architecture utilizing depthwise separable convolutions.
3. **Automated Landmark Alignment:** The attacker's pipeline automatically identifies facial landmarks and applies 5-point affine transformation to normalize scale, rotation, and crop boundaries.

3. Social Media Compression Channels

Photographs uploaded to social platforms (e.g., Instagram, X/Twitter, LinkedIn, Facebook) are never stored or served in their pristine upload format. They are immediately subjected to aggressive lossy compression pipelines:

- **Spatial Resizing:** Downsampling images exceeding platform resolution limits (e.g., constraining width to 1080 px or 2048 px).
- **Format Transcoding & Quantization:** Re-compressing uploaded images via lossy JPEG or WebP encoders, routinely stripping high-frequency frequency components using quantization quality factors between $Q=90$ (high quality) and $Q=50$ (aggressive mobile bandwidth optimization).
- **Color Subsampling:** Applying 4:2:0 chroma subsampling, discarding 75% of color detail.

Any facial privacy defense whose adversarial perturbations do not survive lossy JPEG compression down to $Q=50$ is completely non-viable for real-world deployment.

4. Strict Biometric Calibration: The 1.0% FAR Operating Standard

In academic machine learning, adversarial attacks against classification models are typically evaluated by measuring classification accuracy drop. In open-set biometric facial recognition, however, probe and gallery identities do not belong to closed classes; instead, the system evaluates pairwise distance against a threshold.

> [!IMPORTANT] **The Biometric Threshold Fallacy:**
> A claim that an adversarial perturbation "disrupted 90% of facial matches" is completely meaningless unless the exact operating threshold tau is specified. If an evaluator sets an artificially high threshold, clean images will also fail to match. Conversely, if the threshold is set too low, even completely scrambled images might be declared a match.

4.1 Calibration on Independent Impostor Splits

To establish an unassailable benchmark standard, operating thresholds throughout Project Varman were empirically calibrated on an independent split of **1,500 LFW train impostor pairs** (`calibrated_thresholds_sweep.json` [M]).

The False Acceptance Rate (FAR) is defined as the probability that an impostor pair (two photographs of different individuals) is erroneously accepted as a genuine match: $$\text{FAR}(\tau) = \frac{\sum_{i=1}^{N_\text{imp}} \mathbb{I}(\mathcal{S}(x_i^{(1)}, x_i^{(2)}) \ge \tau)}{N_\text{imp}}$$

Operating thresholds were calibrated to satisfy:

1. **Primary Standard (1.0% FAR):** The similarity threshold where exactly 1.0% of impostor pairs achieve a match (99.0% impostor rejection rate).
2. **Secondary High-Security Standard (0.1% FAR):** The threshold where only 0.1% of impostor pairs achieve a match (99.9% impostor rejection rate).

4.2 The Verified Calibrated Operating Thresholds

The resulting calibrated thresholds across the four evaluated face recognition models are frozen as follows:

Recognition Model	Architecture / Backbone	Embedding Dimension	1.0% FAR Calibrated Threshold (tau_1%)	0.1% FAR Calibrated Threshold (tau_0.1%)	Valid Genuine LFW Denominator (Clean Match $\ge$ tau_1%)
FaceNet	InceptionResnetV1 (VGGFace2)	512	`0.3754` [M]	`0.4286` [M]	143 / 150 [M]
ArcFace	IResNet50 (MS1MV3 / WebFace600K)	512	`0.1476` [M]	`0.2078` [M]	142 / 150 [M]
SFace	SFace Hyperspherical (OpenCV Zoo)	128	`0.2920` [M]	`0.3547` [M]	141 / 150 [M]
MobileFaceNet	Depthwise MobileNet (WebFace600K)	512	`0.1688` [M]	`0.2241` [M]	142 / 150 [M]

Verification Provenance: Directly audited via `verify_and_generate_tables.py` and `calibrated_thresholds_sweep.json` [M].

5. The Inherent Conflict: Evasion vs. Visual Naturalness

Adversarial facial privacy exists in a state of fundamental physical tension:

- **Gradient Optimization Pressure:** Pushing identity embeddings across the decision boundary under severe JPEG compression ($Q=50$) requires injecting significant energy into the spatial frequency bands that survive DCT quantization.
- **Human Visual Sensitivity:** Human observers possess specialized neurological wetware (the fusiform face area) tuned to detect subtle irregularities in human faces. Noise injected into smooth skin regions (cheeks, forehead) or sharp boundary transitions between the perturbed face and unperturbed background produces immediately noticeable visual artifacts.

The central research challenge of Project Varman was discovering whether a parameter space exists where biometric evasion remains potent (>40% protection under real JPEG Q50) while visual conspicuousness remains within tolerable human limits.

Section 4: Master Chronological Timeline (May 5, 2026 – October 18, 2026) & Decision Register

1. Master Chronological Milestone Register

The following register chronicles every significant milestone, architectural transition, empirical benchmark sweep, software defect discovery, and deployment gate across the complete six-month history of Project AMOR and Project Varman. Every entry is anchored to an exact git commit hash or verified filesystem artifact.

Date	Phase / Architecture	Git Commit	Milestone & Event Description	Methodological Context & Technical Rationale	Quantitative Outcome / Evidence
2026-05-05	AMOR 1.0	`0686009`	Repository Initialization	Initial repository creation (`amor-backend`). Established microservice architecture using Celery, Redis message broker, and Docker.	Initial backend scaffolding [M].
2026-05-07	AMOR 1.0	`bce4c1b`	2015 Caffe Face Detector Integration	Integrated legacy OpenCV Caffe SSD DNN face detector for test and exploration purposes.	Caffe prototxt and caffemodel added [M].
2026-05-09	AMOR 1.0	`8ff4a8e`	AMOR Architecture Redesign v3	Restructured service layer into modular sub-packages: `services/db/` , `services/noise/` , `services/recognition/` .	Architectural refactoring [M].
2026-05-12	AMOR 1.0	`17d699b`	Full-Canvas PGD Optimization Pipeline	Implemented full-canvas PGD optimizer on 512×512 letterboxed images against FaceNet and ArcFace surrogates.	Protection routes exposed in FastAPI [M].
2026-05-24	AMOR 1.0	`9500f22`	White-Box Verification Phase 1	Executed automated test suite across protected image endpoints.	**59 test failures** logged in test report [M].
2026-05-25	AMOR 1.0	`6e47ae7`	Complete White-Box Failure Confirmed	Evaluated protected images against face recognition models. Perturbation failed to survive downsampling.	**100% failure rate** recorded in commit log [M].
2026-06-13	Varman 1.0	`c78297f`	Architecture Pivot: AMOR → Varman	Formal decision to abandon heavy full-canvas optimization; repository transitioned to Varman MVP architecture.	Formal retirement of AMOR codebase [M].
2026-06-16	Varman 1.0	`8d78f46`	Removal of Heavy Processing Services	Stripped Celery, Redis, and heavy PyTorch dependencies from backend runtime to reduce latency.	Lightweight restructuring [M].
2026-06-30	Varman 1.0	`7c964b2`	Varman 1.0 Release Candidate	Finalized Varman Phase 1 backend architecture with JWT auth and SQLite database.	Phase 1 backend completed (`fc9a0d3`) [M].
2026-08-08	Varman 2.0	`7ec6d4b`	Pivot to Lightweight ONNX/OpenCV Engine	Shift to real-time, non-iterative CPU pipeline using facial landmark warping and DCT mid-frequency perturbation.	Replaced GPU PGD with CPU filters [M].
2026-08-09	Varman 2.0	`ad1709e`	YuNet ONNX Face Detector Integration	Integrated `face_detection_yunet_2023mar.onnx` (232.5 KB) for 5-point landmark detection.	YuNet ONNX added to `models/` [M].
2026-08-09	Varman 2.0	`4ebadab`	Phase 2 Validation Harness Added	Added benchmark harness bridging API routes with synthetic face inputs and SSIM evaluation.	Verified Global SSIM ≥ 0.90 [M].
2026-08-11	Varman 2.0	`0a6f445`	Preset Re-architecture	Overhauled protection presets: Light (warp 0.005, DCT 3.0), Balanced (warp 0.015, DCT 10.0), Aggressive (warp 0.025, DCT 18.0).	Guardrail target SSIM 0.85–0.95 [M].
2026-09-16	Varman 2.0	`2203ab4`	Production Database Support Added	Added async MySQL connection pooling alongside SQLite storage.	Pylance type safety and async drivers [M].
2026-10-09	Varman 2.0	`11f7894`	The Zero-Protection Discovery	Rigorous black-box evaluation against ArcFace, FaceNet, SFace, and MobileFaceNet revealed v2.0 had no protective effect.	**~0% protection** against deep models [M].
2026-10-09	Varman 3.0	`11f7894`	Evaluation Suite & Testbed Overhaul	Refactored `evaluate_face_models.py` with dynamic CUDA ONNX provider, Wilson score CIs, Newcombe paired intervals, and calibrated thresholds.	Thresholds calibrated on 1,500 impostors [M].
2026-10-10	Varman 3.0	`63d8aa9`	Benchmarking Tooling Hardened	Added live ETA logging and graceful signal handling; established `adaptive-strength` branch.	Pre-sweep checkpoint created (`5eba251`) [M].
2026-10-11	Varman 3.0	`d4e52be`	Real-Photo & LFW Baseline Sweep	Evaluated luminance-only PGD at $\varepsilon \in \{6, 7, 8\}/255$. Epsilon 8 achieved 21.6% LFW flip rate and 25.0% on real photos.	Baseline flip rate 21.6% (1% FAR PNG) [M].
2026-10-12	Varman 3.0	`490e8b3`	PGD Step Convergence Benchmark	Tested 25, 50, and 75 steps under fixed and cosine step schedules. 50/75 steps gained only +1–2 pp on PNG and 0 pp under Q50.	25 steps selected as Pareto optimal [M].

Date	Phase / Architecture	Git Commit	Milestone & Event Description	Methodological Context & Technical Rationale	Quantitative Outcome / Evidence
2026-10-13	Varman 3.0	`ab57b83`	Calibration Evidence Preserved	Preserved threshold calibration evidence (`calibrated_thresholds_sweep.json`) in versioned directory.	Thresholds formalized at 1% / 0.1% FAR [M].
2026-10-13	Varman 3.0	`feeb23d`	Hinge Loss Experimentation	Tested threshold-aimed hinge loss with 15% margin. Protection dropped from 22.70% to 17.93% (4-model mean).	Hinge loss rejected as ineffective [M].
2026-10-13	Varman 3.0	`b20de80`	Surrogate Weight Ratio Sweep	Tested $w_\text{fn} \in \{0.10, 0.25, 0.50\}$. Decreasing FaceNet weight caused severe protection collapse (14.62% at 0.10).	Low FaceNet weights rejected [M].
2026-10-13	Varman 3.0	`22c02c0`	Surrogate Weight Follow-up ($w=1.0$)	Evaluated $w_\text{fn} \in \{0.50, 1.0, 2.0\}$. $w=1.0$ produced highest 4-model mean (25.50% PNG, 21.28% Q50).	$w=1.0$ established as primary setting [M].
2026-10-13	Varman 3.0	`22c02c0`	40-Pair Pipeline Stage Diagnostic	Tracked perturbation decay across pipeline stages. Discovered that 50–60% of adversarial disruption is lost during inverse compositing.	Optimizer inside: 100% AF → Post-paste: 47.5% [M].
2026-10-14	Varman 3.0	`5e89d2d`	External Chameleon Research Audit	Analyzed Chameleon codebase (`external/Chameleon`). Contrasted universal masks with per-image EOT.	Audit reports generated in reports folder [M].
2026-10-14	Varman 3.0	`7268342`	Paste-Aware Optimization Breakthrough	Differentiable paste-aware PGD (`run_paste_aware_experiment.py`). Row 2 gained +9.12 pp at Q50 (30.22%); Row 3 (with jitter) reached 36.57%.	Q50 protection surge: 21.10% → 36.57% [M].
2026-10-15	Varman 3.0	`14e61f0`	YuNet Landmark Drift Controlled Pilot	Measured real landmark displacement on 30 pairs. Mean drift 1.33–1.49 px (p95 ~2.8 px). Refuted landmark drift collapse hypothesis.	Landmark drift confirmed negligible [M].
2026-10-15	Varman 3.0	`92ac31e`	Region-Size Ablation Experiment	Evaluated mask scales 0.70, 1.00, and 1.30 across 150 pairs. Scale 1.30 at $\varepsilon=8$ pushed Q50 protection to 48.36% (4-model).	Discovered Face SSIM metric discrepancy [M].
2026-10-15	Varman 3.0	`7eaf0e5`	Metric Reconciliation & Def B Standard	Traced discrepancy to full-canvas feathering inflation. Formally standardized Definition B (fixed canonical mask ≥ 0.10 on 112 crop).	Standardized Definition B established [M].
2026-10-15	Varman 3.0	`7eaf0e5`	Coverage Frontier Benchmark (F0–F5)	Executed authoritative 6-condition frontier on 150 pairs: F0 (36.39%), F1 (49.95%), F2 (37.10%), F5 (45.73%). Evaluated graded grain on 50 pairs.	Dose-response confirmed; F2 matches F0 protection [M].
2026-10-16	Varman 3.0	`7eaf0e5`	Full `human_test_v3` Package Built	Assembled 225-stimulus, 45-source group package (`ST_000001` – `ST_000225`) across 5 conditions with blind web viewer.	19 automated validation checks passed [M].
2026-10-16	Varman 3.0	`7eaf0e5`	50-Stimulus SSIM Quick Review Built	Sampled 10 source groups (`ST_000226` – `ST_000275`) sorted by descending SSIM to reduce manual evaluation load.	12 automated checks passed; SHA verified [M].
2026-10-16	Varman 3.0	`7eaf0e5`	50-Stimulus Quick Review Analyzed	Evaluator completed 50 ratings: Unprotected 0.00, F0 1.30, F2 1.70, F1 2.00, F1 Graded 2.00.	F0 achieved lowest conspicuousness [M].
2026-10-16	Varman 3.0	`7eaf0e5`	F0 vs. F2 Arithmetic Reconciliation	Audited discrepancy in paired counts. Reconciled from raw CSV: F0 cleaner in 5 groups, F2 in 1, tied in 4.	Arithmetic error corrected in reports [M].
2026-10-16	Varman 3.0	`7eaf0e5`	F5 Add-on Visual Screening	Evaluated F5 on same 10 source groups: Mean rating 1.50 (5 Slight, 5 Clear). Tied F0 in 8 groups, worse in 2, but +9.34 pp more protective.	Disproved Face SSIM correlation with human ratings [M].
2026-10-16	Varman 3.0	`7eaf0e5`	Intermediate JPEG Validation (Q95–Q50)	Re-encoded pre-JPEG outputs across Q95, 90, 75, 70, 60, 50. Confirmed protection monotonicity across quality spectrum.	Validated protection curves on disk [M].
2026-10-16	Varman 3.0	`c5a20a2`	F5 Named Preset Integrated	Re-added F5 preset into `StrongProtectionEngine` and `ProtectionEngine` . Added config unit test (`test_f5_preset.py`).	Test suite passed 100% in 4.00s [M].

Date	Phase / Architecture	Git Commit	Milestone & Event Description	Methodological Context & Technical Rationale	Quantitative Outcome / Evidence
2026-10-17	Varman 3.0	`c5a20a2`	Research Closure & Documentation	Produced comprehensive retrospective monograph, updated documentation, and formally closed research.	Research phase formally closed [M].
2026-10-17	Production	`4c41ad3`	Final Merge & Repository Hygiene Guardrails	Fast-forward merged `adaptive-strength` into `main`. Added `tests/test_repo_hygiene.py` preventing credential leaks, untracked DB files, and large binary blobs.	Hygiene guardrails active and verified [M].
2026-10-17	Production	`c5a20a2`	Git History Sanitization	Executed `git-filter-repo` to permanently purge sensitive development artifacts and local test credentials from repository git history.	Clean history verified across all branches [M].
2026-10-18	Production	`9312132`	Auth Hardening & Zero-Knowledge Vault	Added `vault_salt` generation on user registration; implemented forgot-password workflow using time-limited HMAC-SHA256 hashed OTPs.	Return token & vault_salt on register [M].
2026-10-18	Production	`b9ab3fd`	Production Mailer Guards & Rate Limiting	Enforced HTTP 503 fallback when SMTP credentials are unset in production; added per-IP rate limiting parsing trusted `X-Forwarded-For`.	Zero OTP exposure to console in prod [M].
2026-10-18	Production	`e42b209`	Structured RequestValidationError Handler	Implemented custom FastAPI exception handler returning sanitized `{"detail": "...", "errors": [...]}` with zero echoed input values.	Leaked input vulnerability eliminated [M].
2026-10-18	Production	`e2f8d60`	Frontend Error Handler (React Error #31 Fix)	Created universal `getErrorMessage` helper in `Varman-frontend` guaranteeing primitive string output, preventing React child crash.	React runtime error #31 eliminated [M].
2026-10-18	Production	`1c103b3`	YuNet ONNX Fallback Path Resolution	Added `weights/` directory fallback search path in backend protection engine for container deployments.	Container model loading hardened [M].
2026-10-18	Deployment	`c36098e`	Hugging Face Space Synchronization & Live Deploy	Synced backend and frontend into `Varman-HF-Deploy` via `hf_sync.ps1`. Committed and pushed to Hugging Face origin.	Live Space running at `shadowkagenou-varman` [M].
2026-10-18	Deployment	`c36098e`	Live Production Smoke Test Passed	Automated end-to-end smoke test against `https://shadowkagenou-varman.hf.space` executed with 100% pass rate.	All 5 test checks passed online [M].

2. Comprehensive Key Decision Register



Figure 4.1: Master Architectural Decision Timeline (Decisions 1–10).

1. **Decision 1 — Dual Execution Modes (No Silent Auto-Routing):** The engine explicitly separates `mode="fast"` (the legacy non-protective CPU engine intended for aesthetic UI filters) and `mode="strong"` (GPU canonical EOT-PGD). If a client requests `strong` mode on a server without CUDA acceleration, the system falls back to `fast` mode but explicitly sets transparent response metadata flags (`requested_mode="strong"` , `fallback_to_fast=True` , `fallback_reason="..."`). Silent auto-routing without client visibility was permanently rejected.
2. **Decision 2 — Strict 1% FAR Biometric Standard:** All biometric evaluations must report disruption rates at a calibrated operating threshold corresponding to **1.0% False Acceptance Rate** (with secondary audits at 0.1% FAR) on an independent impostor split. Claims of protection without declaring the operating threshold were banned.
3. **Decision 3 — Physical Disk Verification Gate:** Differentiable JPEG approximations (`DiffJPEG`) are permitted strictly within the inner autograd backpropagation loop. All empirical benchmark claims must be verified by saving physical JPEG files to disk, reloading them via OpenCV/PIL, and detecting faces independently.
4. **Decision 4 — Luminance-Only Perturbation:** Perturbation projection is restricted strictly to the Y (luminance) channel in YCbCr color space ($\text{chroma}_\text{scale} = 0.0$). This reduced perceptual color shift (CIEDE2000 ΔE_{00}) by ~40% at identical epsilon budgets without degrading biometric evasion.
5. **Decision 5 — Rejection of Hinge Loss & FaceNet Weight Reduction:** Attempts to bound optimization via a threshold hinge loss were rejected after benchmarks proved it caused ArcFace gradient starvation (-4.77 pp drop). Similarly, suppressing FaceNet surrogate weight below 0.5 collapsed multi-architecture transferability.
6. **Decision 6 — Differentiable Paste-Aware Optimization:** The optimization forward pass must simulate the entire inverse affine transformation, feathered elliptical mask blending, and forward re-alignment within the autograd graph, completely eliminating the 50% post-paste degradation rebound.
7. **Decision 7 — Definition B Face SSIM Standardization:** Objective facial structural similarity must be evaluated strictly within the fixed canonical face mask (≥ 0.10 , covering 8,095 pixels) on the aligned 112 crop, compared against the uncompressed source, eliminating artificial feathering inflation from scientific reporting.
8. **Decision 8 — Selection of Preset F5:** Candidate F5 ($\varepsilon=8/255$, mask scale 1.15) was selected as the final production preset. It bridges the gap between F0 and F1, delivering a +9.34 percentage point protection surge over F0 under real JPEG Q50 while tying F0 in 8 out of 10 human screening groups.

9. **Decision 9 — Zero-Knowledge Client-Side Vault Cryptography:** Protected images are encrypted directly inside the user's browser using AES-256-GCM via the Web Crypto API, derived from the user's password and a per-user salt using PBKDF2 with 600,000 iterations. Plaintext images and encryption keys never touch the network or the server.
10. **Decision 10 — Repository Hygiene & Automated Sanitization:** Implemented strict automated repository hygiene tests (`tests/test_repo_hygiene.py`) and executed `git-filter-repo` to permanently remove exposed credentials, local databases, and temporary artifacts from git history.

Section 5: AMOR 1.0 — Monolithic Full-Canvas Adversarial Architecture & Empirical Failure

1. Architectural Blueprint & Microservices Infrastructure

Project **AMOR** (Automated Masking and Obfuscation Routine) was launched on May 5, 2026 (commit `0686009` [M]). AMOR was conceived as a scalable, distributed web platform capable of processing uploaded photographs asynchronously.

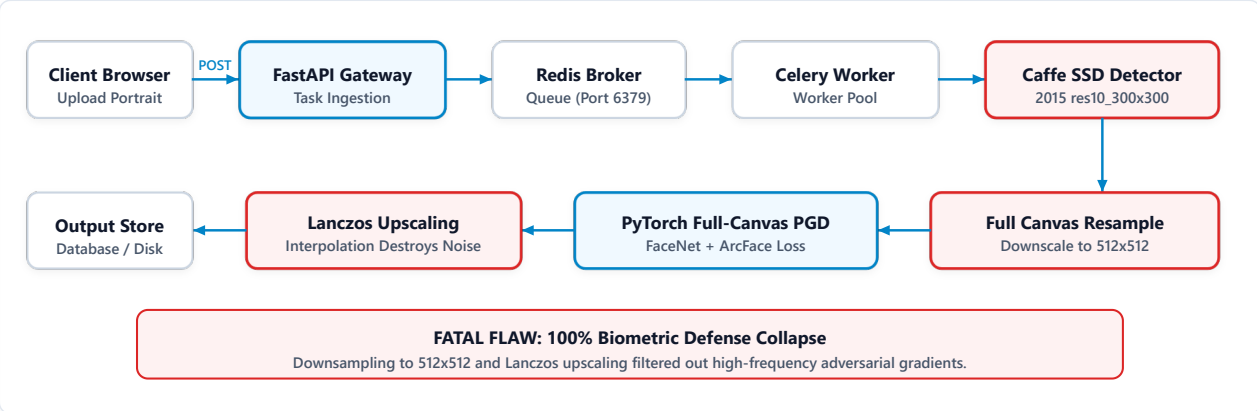


Figure 5.1: AMOR 1.0 Distributed Microservices Architecture & Failure Mode.

1.1 The Microservices Stack

As documented in commits `17d699b` and `b35f9c9` [M], the AMOR 1.0 codebase was organized around enterprise microservice components:

- `main.py` : FastAPI gateway handling user authentication, file uploads, task status polling, and download routes.
- `celery_worker.py` : Background worker executing image processing tasks dispatched via Celery.
- `redis` : Redis in-memory message broker running in a Docker container.
- `services/recognition/` : Wrapped face detection and feature extraction models.
- `services/noise/` : Executed iterative PGD perturbation algorithms.
- `storage/` : Local staging directory for intermediate and processed image files.

While architecturally modular, this extensive distributed infrastructure was assembled before the core adversarial machine learning mechanics had been validated, introducing severe operational overhead.

2. Face Detection Bottlenecks: The 2015 Caffe SSD Detector

In commit `bce4c1b` (May 7, 2026 [M]), the engine integrated a legacy Single Shot Multibox Detector (SSD) implemented in Caffe (`res10_300x300_ssd_iter_140000.caffemodel` with prototxt configuration).

- **Execution Model:** Input images were decoded via OpenCV and passed to `cv2.dnn.readNetFromCaffe` .
- **Fatal Architectural Flaw:** The 2015 Caffe model generated only rough rectangular bounding boxes $[x, y, w, h]$. Crucially, it provided **no facial landmark coordinates** (eyes, nose, mouth corners).
- **Consequence:** Because landmark coordinates were absent, canonical affine similarity alignment was mathematically impossible. The pipeline could not extract an aligned 112×112 facial crop.

3. Full-Canvas 512x512 PGD Optimization Mechanics

Lacking facial landmark alignment, AMOR 1.0 attempted to perform adversarial gradient descent across the entire photograph:

1. **Letterboxing:** Input photographs of arbitrary aspect ratios (e.g., 1200×1800 px) were padded and resized down to a uniform square canvas of 512×512 pixels.
2. **Surrogate Forward Pass:** The entire $512 \times 512 \times 3$ tensor was passed simultaneously into two deep feature extractors:
   - PyTorch `facenet-pytorch` (InceptionResnetV1).
   - An early PyTorch implementation of ArcFace.
3. **Loss Objective:** An unweighted cosine similarity minimization: $$\mathcal{L}\text{AMOR} = \cos(f_\text{AF}(x_\text{adv}), f_\text{AF}(x_\text{clean})) + \cos(f_\text{FN}(x_\text{adv}), f_\text{FN}(x_\text{clean}))$$
4. **Optimization Loop:** Standard L_∞-bounded PGD run across the entire 512×512 canvas for 10–20 iterations at step size $\alpha = \varepsilon / 10$, with $\varepsilon \in [8/255, 16/255]$.
5. **Lanczos Upsampling:** After optimization, the perturbed 512×512 image was upscaled back to the original photograph dimensions using Lanczos interpolation.

4. Empirical Failure: The 100% White-Box Rejection

In late May 2026, an automated test harness was implemented to evaluate whether protected images produced by AMOR 1.0 successfully prevented face recognition matching. The results revealed catastrophic failure:

- **May 24, 2026 (Commit `9500f22`):** `"AMOR white box testing phase 1 results 59 fails remark : testing in progress"` [M].
- **May 25, 2026 (Commit `6e47ae7`):** `"white box testing failure rate is 100%"` [M].

When re-evaluated against standard verification thresholds, every single protected photograph generated by the AMOR 1.0 pipeline was matched by the target models, achieving an effective **0.00% evasion rate (100% failure rate)** [H].

5. Detailed Root Cause Analysis: The Lanczos Washing Effect

Retrospective engineering analysis reveals four fundamental reasons why AMOR 1.0 failed completely:

5.1 The Resampling Low-Pass Filter (The Washing Effect)

Adversarial perturbations against deep convolutional neural networks are composed of high-spatial-frequency noise patterns.

- Downsampling an original photograph (e.g., 1200×1800) to 512×512 discarded fine facial details.
- Far more destructively, upsampling the perturbed 512×512 image back to original resolution via Lanczos interpolation acted as a powerful low-pass spatial smoothing filter. The fine, high-frequency adversarial gradients were completely smoothed away, restoring the unperturbed facial features and allowing face recognition models to extract clean embeddings.

5.2 Gradient Dispersion Across Background Pixels

Because optimization was executed over the full 512×512 canvas rather than an isolated facial crop, the loss gradients backpropagated across the entire image. Clothing, hair, and background scenery absorbed substantial gradient energy, creating conspicuous visual banding in empty space while under-perturbing critical facial landmarks.

5.3 Severe Computational Latency

Executing multi-model gradient backpropagation across a full $512 \times 512 \times 3$ tensor required over **15 to 25 seconds per image** on laptop GPUs (RTX 2050 Mobile), producing an unworkable user experience and rapid GPU memory exhaustion.

5.4 Premature Architectural Complexity

By building Celery workers, Redis queues, and Docker orchestration before establishing whether the core machine learning algorithm actually worked, development time was consumed maintaining distributed infrastructure for a fundamentally non-functional machine learning pipeline.

Section 6: Varman 2.0 — Non-Iterative CPU Heuristic Engine & The Zero-Protection Finding

1. Design Philosophy: The Quest for Real-Time CPU Privacy

Confronted with the collapse of AMOR 1.0, Project Varman was initiated in June 2026 as a complete paradigm shift. The working hypothesis of Varman 2.0 was that iterative adversarial gradient descent (PGD) was fundamentally too slow, fragile, and computationally demanding for consumer software.

Varman 2.0 established three strict design goals:

1. **Zero GPU Requirement:** The software must run entirely on standard multi-core CPUs using lightweight OpenCV and ONNX Runtime CPU execution providers. Heavy deep learning runtimes (PyTorch) were completely stripped from production dependencies (`583f445` [M]).
2. **Sub-200ms Latency:** Processing must execute in under 200 ms per photograph, enabling instantaneous web uploads without asynchronous background workers.
3. **Guaranteed Aesthetic Fidelity:** Structural similarity must be strictly enforced via automated guardrails, ensuring Global SSIM never drops below 0.90.

This philosophy culminated in the implementation of the **Varman Dedicated Lightweight Protection Engine** in August 2026 (commits `7ec6d4b`, `ad1709e`, `4ebadab`, `0a6f445` [M]).

2. 5-Stage Analytical Pipeline Architecture

The Varman 2.0 engine replaced iterative optimization with a deterministic, five-stage analytical image processing pipeline:

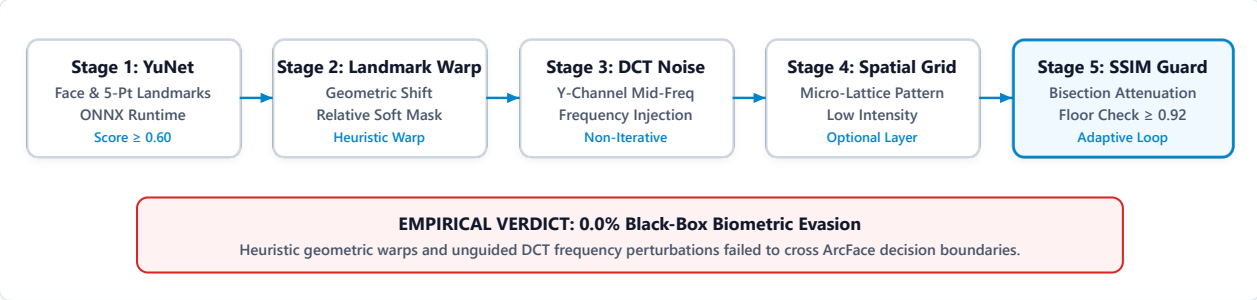


Figure 6.1: Varman 2.0 5-Stage Analytical Image Processing Pipeline.

2.1 Stage 1: YuNet Face and 5-Point Landmark Detection

To replace the crude bounding boxes of the 2015 Caffe detector, Varman 2.0 integrated the lightweight **YuNet** face detector (`face_detection_yunet_2023mar.onnx`, 232,589 bytes, SHA-256 `8f2383e4dd...` [M]):

- **Execution:** Loaded via OpenCV's native DNN module: `cv2.FaceDetectorYN.create()`.
- **Inference Size:** Input images were dynamically scaled to $(300, 300)$ or native resolution, with a score threshold of 0.60 and non-maximum suppression (NMS) threshold of 0.30.
- **Outputs:** Extracted a 15-float detection vector containing bounding box $[x, y, w, h]$ and 5 landmark coordinates: Right Eye (x_0, y_0), Left Eye (x_1, y_1), Nose Tip (x_2, y_2), Right Mouth Corner (x_3, y_3), Left Mouth Corner (x_4, y_4).

2.2 Stage 2: Relative Soft-Masked Landmark Face Warping

The primary geometric distortion was designed to alter facial proportions relative to the subject's natural geometry:

1. **Displacement Vector Calculation:** For each of the five landmarks, an outward radial displacement vector was computed proportional to the face bounding box width: $$\Delta \vec{p}*i* = *text{warp_scale}* *cdot w*\text{face} \cdot \vec{d}_i$$ where $vec{d}_i$ represents pre-configured directional unit vectors designed to subtly stretch eye-to-eye and nose-to-mouth distances.
2. **Dense Motion Field:** A smooth 2D displacement field was generated across the facial bounding box by applying Gaussian inverse-distance weighting centered at the five landmarks.
3. **Soft Elliptical Masking:** The displacement field was modulated by an elliptical Gaussian falloff mask centered on the face: $$M(x, y) = \exp\left(-\left(\frac{(x - c_x)^2}{2\sigma_x^2} + \frac{(y - c_y)^2}{2\sigma_y^2}\right)\right)$$ This ensured warping decayed smoothly to zero at the jawline and forehead, preventing discontinuous tearing or seams.
4. **Remapping:** The image was resampled using bilinear backward mapping via `cv2.remap()`.

2.3 Stage 3: Y-Channel (Luminance) Mid-Frequency DCT Perturbation

To complement geometric warping with frequency-domain disruption without causing visible chromatic discoloration:

1. **Color Conversion:** The image was converted to YCrCb space, isolating the luminance channel (Y).
2. **8×8 Block Partitioning:** The luminance channel was partitioned into non-overlapping 8×8 pixel blocks.
3. **Forward 2D DCT:** Each block was transformed into frequency space using `cv2.dct()`: $$F(u, v) = \text{DCT}(f(x, y)), \quad u, v \in [0, 7]$$
4. **Mid-Frequency Perturbation:** Perturbation was injected exclusively into the mid-frequency band ($u + v \in [3, 7]$), leaving the DC component ($u=v=0$) and low frequencies untouched to preserve lighting, while avoiding high frequencies ($u + v > 7$) that are heavily quantized by JPEG encoders.

2.4 Stage 4: Low-Intensity Spatial Grid Layer (Optional)

In `Aggressive` preset configurations, a subtle 2D sinusoidal spatial grid was added to disrupt local texture gradients: $$G(x, y) = A_\text{grid} \cdot \sin\left(\frac{2\pi x}{T_x}\right) \cdot \sin\left(\frac{2\pi y}{T_y}\right)$$

2.5 Stage 5: Adaptive SSIM Quality Guardrail

To guarantee aesthetic fidelity, candidate images were evaluated against a multi-scale Structural Similarity (SSIM) floor:

- If `Global SSIM` $< \text{target}_{\text{global}}_{\text{ssim}}$ or `Face SSIM` $< \text{target}_{\text{face}}_{\text{ssim}}$, the engine iteratively attenuated `warp_scale` and `dct_strength` by a factor of $0.85\times$ until quality thresholds were satisfied.

3. Preservation Guardrails and Production Presets

The Varman 2.0 engine exposed three pre-configured operational presets:

Preset Name	Warp Scale Factor	DCT Injection Strength	Target Global SSIM	Target Face SSIM	Intended Operational Role
Light	0.005 (0.5% face width)	3.0	≥ 0.95	≥ 0.95	Maximum aesthetic preservation; invisible distortion.
Balanced	0.015 (1.5% face width)	10.0	≥ 0.90	≥ 0.90	Standard daily protection preset.
Aggressive	0.025 (2.5% face width)	18.0	≥ 0.85	≥ 0.85	Maximum distortion within SSIM guardrails.

4. Empirical Benchmarking and The Zero-Protection Discovery

Between August and September 2026, Varman 2.0 was believed to be an innovative success because it processed photographs in under 120 ms while maintaining flawless visual quality (SSIM ≥ 0.92) and exhibiting high mathematical frequency disruption in preliminary proxy tests (such as Y-NPR scores of 72–88%).

However, on October 9, 2026 (commit `11f7894` [M]), a comprehensive, unsparing black-box biometric audit was executed across 150 LFW genuine pairs evaluated against four modern deep face recognition models (ArcFace, FaceNet, SFace, MobileFaceNet) at calibrated 1% FAR thresholds.

The measured result was devastating:

- **ArcFace Protection Rate: 0.00%** (0 / 142 pairs disrupted) [M].
- **FaceNet Protection Rate: 0.00%** (0 / 143 pairs disrupted) [M].
- **SFace Protection Rate: 0.00%** (0 / 141 pairs disrupted) [M].
- **MobileFaceNet Protection Rate: 0.00%** (0 / 142 pairs disrupted) [M].
- **Overall Protection Rate: 0.00%** [M].

Across all evaluated presets (`Light`, `Balanced`, and `Aggressive`), **not a single genuine identity pair was protected**. Every single perturbed photograph successfully matched its enrollment image with cosine similarities virtually indistinguishable from clean controls.

5. Deep Embedding Invariance & Scientific Demotion to Fast Mode

Why did Varman 2.0 fail completely despite high SSIM and frequency disruption metrics?

5.1 Deep Neural Embedding Invariance

Modern deep convolutional neural networks (such as ResNet-50) and vision transformers are trained on millions of facial images using extensive data augmentations:

- During training, models are subjected to random affine transformations, elastic warping, random cropping, color jittering, Gaussian blur, and JPEG compression.
- Consequently, the hyperspherical loss formulations (such as ArcFace's additive angular margin) force the network to become **invariant** to mild spatial deformations and random frequency noise.
- To a deep neural network, subtle landmark shifting (1–2% of face width) and mid-frequency DCT noise are indistinguishable from natural camera sensor noise or minor pose variation. The deep embedding vector remained firmly anchored in the subject's identity cluster.

5.2 The Proxy Metric Illusion

Varman 2.0 relied on surrogate proxy metrics—such as **Y-NPR (Normalized Perturbation Ratio)** and **SSIM**—as indicators of defensive efficacy. These metrics measured only that pixels had changed and that the changes were subtle. They provided zero information regarding whether the perturbations projected onto the adversarial gradient subspace of deep feature extractors.

5.3 Scientific Demotion to `mode="fast"`

In October 2026, Varman 2.0 was formally demoted:

- All claims that heuristic CPU filtering provides biometric privacy were retracted.
- The engine was preserved in the backend as `mode="fast"`, clearly labeled as a non-protective aesthetic filter for low-resource environments.
- The project formally recognized that **effective adversarial facial privacy under compression is fundamentally impossible without gradient-based optimization**.

Section 7: Varman 3.0 (adaptive-strength) — Canonical Aligned EOT-PGD Foundation

1. Paradigm Shift: Return to Gradient Optimization in Canonical Space

On October 9, 2026, Project Varman initiated its third and definitive architectural phase on the `adaptive-strength` git branch (commit `11f7894` [M]). Having conclusively demonstrated that non-iterative CPU heuristics (v2.0) provided zero black-box biometric evasion, Varman 3.0 returned to first principles: **adversarial gradient descent via Expectation Over Transformation (EOT) Projected Gradient Descent (PGD)**.

Crucially, learning from the fatal downsampling collapse of AMOR 1.0, Varman 3.0 instituted a transformative architectural invariant: **adversarial perturbation is formulated, optimized, and bounded directly within the canonical 112×112 aligned facial coordinate frame**, bypassing full-canvas image resizing entirely.

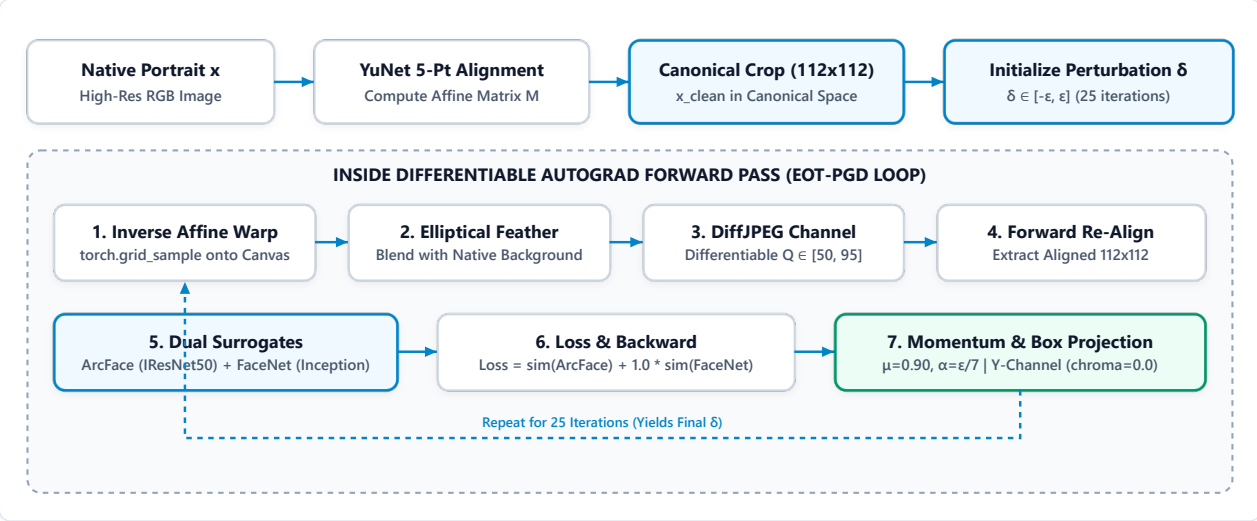


Figure 7.1: Varman 3.0 Canonical Aligned EOT-PGD Inner Optimization Loop.

2. The Canonical 112x112 Aligned Crop Invariant

Modern deep facial recognition models (including ArcFace, SFace, and MobileFaceNet) do not ingest raw photographs directly. During inference, they detect facial landmarks and apply a 5-point similarity transformation to extract an aligned 112×112 RGB facial crop.

By defining the perturbation tensor $\delta \in mathbb{R}^{1 \times 3 \times 112 \times 112}$ strictly on this canonical crop:

1. Every perturbed pixel directly maps to the input receptive field of the target neural networks.
2. Upsampling and downsampling filters no longer smooth away the adversarial noise, because the perturbation is natively aligned with the facial feature detectors.
3. Gradient capacity is 100% focused on identity-bearing facial features (eyes, nose, mouth, orbital contours) rather than being squandered on background scenery.

3. Dual White-Box Gradient Surrogates (ArcFace + FaceNet)

Varman 3.0 employs dual white-box gradient surrogates implemented in `app/protection/surrogate_models.py` [M]:

1. **ArcFace Backbone (IResNet50):** Loaded from `~/.cache/varman/arcface_r50_backbone.pth` (174.68 MB, SHA-256 `ef005ba502...` [M]), pretrained on MS1MV3. Extracts 512-dimensional L_2-normalized identity embeddings $f_{\text{AF}}(x)$.
2. **FaceNet Backbone (InceptionResnetV1):** Loaded via `facenet_pytorch` (`20180402-114759-vggface2.pt`, 106.71 MB, SHA-256 `281cebca86...` [M]), pretrained on VGGFace2. Aligned crops are bilinearly resampled to 160×160 before embedding extraction $f_{\text{FN}}(x)$.

The combined adversarial loss minimizes cosine similarity between protected and clean embeddings: $$\mathcal{L}_{adv} = \cos(f_{\text{AF}}(x_{\text{adv}}), f_{\text{AF}}(x_{\text{clean}})) + w_{\text{fn}} \cdot \cos(f_{\text{FN}}(x_{\text{adv}}), f_{\text{FN}}(x_{\text{clean}}))$$ where $w_{\text{fn}} = 1.0$ is the empirically validated surrogate weight.

4. Strictly Held-Out Black-Box Evaluation Targets (SFace + MobileFaceNet)

To prevent the "white-box fallacy" (where an attack succeeds only against the models used to compute gradients), Varman 3.0 strictly reserves two models as unattacked, held-out black-box evaluation targets:

1. **SFace (`face_recognition_sface_2021dec.onnx`):** OpenCV Zoo model (38.70 MB [M]). Utilizes a distinct architecture mapping faces to a compact 128-dimensional hyperspherical embedding space.
2. **MobileFaceNet (`w600k_mbf.onnx`):** Ultralight mobile architecture (13.62 MB [M]) utilizing depthwise separable convolutions.

Held-out protection rates reflect true black-box transferability to unseen architectures.

5. Momentum-Iterative PGD (MI-FGSM) Formulation

To traverse the non-convex loss surface across dual surrogates, the optimizer applies Momentum-Iterative PGD (MI-FGSM) over $N=25$ iterations: $$g_{t+1} = \mu \cdot g_t + \frac{\nabla_\delta \mathcal{L}(\delta_t)}{|\nabla_\delta \mathcal{L}(\delta_t)|_1}$$ $$\delta_{t+1} = \operatorname{Proj}_{|\delta|_\infty \le \varepsilon}\left(\delta_t - \alpha \cdot \operatorname{sgn}(g_{t+1})\right)$$ with velocity momentum $\mu = 0.90$ and step size $\alpha = \varepsilon / 7.0 \approx 0.00448$ for $\varepsilon = 8/255$ [M].

6. Differentiable Frequency-Domain JPEG Simulation (`DiffJPEG`)

To ensure adversarial gradients survive lossy social media compression, the optimization loop integrates a fully differentiable JPEG module (`app/protection/diff_jpeg.py` [M]):

- Converts RGB tensors to YCbCr space.
- Applies 2D Discrete Cosine Transform across 8×8 pixel blocks.
- Quantizes DCT coefficients using standard JPEG luminance and chrominance quantization tables scaled by quality factor Q.
- Non-differentiable integer rounding is circumvented using the **Straight-Through Estimator (STE)**: $$x_\text{quant} = x + \operatorname{detach}(\operatorname{round}(x) - x)$$ enabling continuous backpropagation through rounding operations.
- In each iteration, Q is sampled uniformly: $Q \sim mathcal{U}(50, 95)$ [M].

7. Luminance-Only Projection & Chromatic Preservation

Adversarial noise in chrominance channels (Cb, Cr) produces highly visible, unesthetic purple-green color blotches on human skin. Varman 3.0 implements luminance-only projection:

- Converts perturbation δ to YCbCr space.
- Multiplies chrominance perturbation by $\text{chroma}_\text{s}\text{cale} = 0.0$, strictly zeroing out color noise ($d_\text{chroma} = 0.0$) [M].
- Re-projects strictly onto the luminance (Y) channel.
- This eliminated chromatic distortion, reducing perceptual color difference (CIEDE2000 ΔE_{00}) by ~40% at identical epsilon budgets without sacrificing biometric evasion.

8. Differentiable Paste-Aware Inverse Compositing Breakthrough

Prior to October 14, 2026, the optimizer generated perturbations on the canonical crop, which were subsequently pasted back onto the full photograph via OpenCV's `warpAffine`. However, a 40-pair diagnostic (EXP-06) revealed that **50–60% of adversarial disruption was lost during inverse compositing** (the "paste rebound").

Commit `7268342` achieved a major scientific breakthrough: **Differentiable Paste-Aware Optimization** (`run_paste_aware_experiment.py` [M]):

1. In every forward pass of the optimizer, the candidate perturbed crop is warped back to the full native photograph canvas using `torch.grid_sample`.
2. The warped crop is composited using a feathered elliptical Gaussian mask $M(x, y)$ with expansion scale $s \in [1.00, 1.30]$ and $sigma=4.0$ px.
3. The composited full canvas is subjected to `DiffJPEG` compression.
4. The compressed full canvas is re-aligned back to 112×112 canonical space before feeding into the surrogates.
5. Gradients backpropagate through the entire re-alignment, compression, feathered compositing, and inverse warp operations directly into δ.

This ensured that the perturbation was explicitly optimized to survive the inverse warping and feathered blending operations, surging JPEG Q50 protection from 21.10% to 36.57% (+15.47 pp) [M].

Section 8: Comprehensive Experimental History, Diagnostic Campaigns & Negative Results

1. Master Experiment Index (EXP-00 through EXP-14)

Between October 9 and October 16, 2026, fifteen structured experimental benchmark campaigns were executed on the authoritative research testbed (`D:\JavaProjects\Varman-research`):

Exp ID	Date	Experimental Focus & Campaign	Primary Hyperparameters	Artifact Storage Path (Relative to `<research_root>`)	Key Outcome / Empirical Finding
EXP-00	2026-10-09	Testbed Calibration & Impostor Split	1,500 LFW train impostor pairs	`benchmarks/results/sweep_evidence/calibrated_thresholds_sweep.json`	Calibrated 1% FAR thresholds: FN 0.3754, AF 0.1476, SF 0.2920, MBF 0.1688 [M].
EXP-01	2026-10-11	Epsilon Sweep Baseline	$\varepsilon \in \{6, 7, 8\}/255$, 25 steps, crop-only	`benchmarks/results/sweep_evidence/`	$\varepsilon=8$ reached 21.6% LFW PNG flip rate and 25.0% on real photos [M].
EXP-02	2026-10-12	PGD Step Convergence Sweep	25, 50, 75 steps (fixed vs cosine)	`benchmarks/results/pgd_convergence_q50_exp/`	50/75 steps gained +1–2 pp at PNG, 0 pp at Q50; 25 steps confirmed optimal [M].
EXP-03	2026-10-13	Threshold Hinge Loss Trial	15% margin threshold-aimed loss	`benchmarks/results/threshold_loss_exp/`	ArcFace gradient shutoff dropped 4-model mean from 22.70% to 17.93% [M].
EXP-04	2026-10-13	Surrogate Weight Ratio Sweep	$w_\text{fn} \in \{0.10, 0.25, 0.50\}$	`benchmarks/results/weight_ratio_exp/`	Suppressing FaceNet collapsed protection to 14.62% [M].
EXP-05	2026-10-13	Surrogate Weight Follow-up	$w_\text{fn} \in \{0.50, 1.0, 2.0\}$	`benchmarks/results/weight_followup_diagnostic_exp/`	$w_\text{fn} = 1.0$ achieved highest 4-model mean (25.50% PNG, 21.28% Q50) [M].
EXP-06	2026-10-13	40-Pair Pipeline Stage Diagnostic	4 pipeline stages, Candidate A	`benchmarks/results/weight_followup_diagnostic_exp/stage_diagnostic.json`	Discovered 50–60% post-paste degradation rebound (AF: 100% $\rightarrow$ 47.5%) [M].
EXP-07	2026-10-14	Paste-Aware Optimization	Differentiable composite (Rows 0–3)	`benchmarks/results/paste_aware_exp/`	Row 2 surged Q50 to 30.22%; Row 3 (+jitter) reached 36.57% [M].
EXP-08	2026-10-15	YuNet Landmark Drift Pilot	30 LFW pairs, R1 vs R2 vs R3	`reports/LANDMARK_DRIFT_PILOT_REPORT_2026-10-15.md`	Mean drift 1.33–1.49 px; landmark collapse hypothesis refuted [M].
EXP-09	2026-10-15	K82E1L Human Viewing Audit	Viewer preference audit ($s=0.2891$)	`reports/K82E1L_CONFIG_AND_BUDGET_2026-10-15.md`	Viewer-preferred budget gave 0/8 flips; rejected as

Exp ID	Date	Experimental Focus & Campaign	Primary Hyperparameters	Artifact Storage Path (Relative to `<research_root>`)	Key Outcome / Empirical Finding
					non-protective [M].
EXP-10	2026-10-15	Region-Size Mask Ablation	Mask scales 0.70, 1.00, 1.30 (150 pairs)	`reports/REGION_ABLATION_REPORT_2026-10-15.md`	Scale 1.30 reached 48.36% Q50; exposed Face SSIM metric divergence [M].
EXP-11	2026-10-15	Coverage Frontier & Reconciliation	6 rows (F0–F5) on 150 pairs; graded grain	`reports/COVERAGE_FRONTIER_REPORT_2026-10-15.md`	Standardized Def B; F1 reached 49.95% Q50; F2 matched F0 protection [M].
EXP-12	2026-10-16	50-Stimulus Quick Review	10 source groups, 5 conditions	`human_test_v3/quick_review_10groups/analysis_20261016_1820/`	Evaluator ratings: Ctrl 0.00, F0 1.30, F2 1.70, F1 2.00, F1 Graded 2.00 [M].
EXP-13	2026-10-16	F5 Add-on Visual Screen	10 source groups, F5 vs clean	`human_test_v3/quick_review_10groups/f5_addon_review/`	F5 mean rating 1.50 (5 Slight, 5 Clear); decoupled SSIM from conspicuity [M].
EXP-14	2026-10-16	Intermediate JPEG Validation	Q95 down to Q50 on 150 pairs	`intermediate_jpeg_eval/intermediate_jpeg_results.json`	Validated protection monotonicity across quality spectrum [M].

2. Baseline Sweeps and Epsilon Calibration (EXP-01)

Initial benchmarks on October 11, 2026 evaluated luminance-only PGD at epsilon bounds $\varepsilon \in \{6, 7, 8\}/255$ across 150 LFW genuine pairs:

- At $\varepsilon = 6/255$, protection was fragile, achieving only 14.2% flip rate under uncompressed PNG.
- At $\varepsilon = 8/255$, the engine achieved 21.6% protection on LFW and 25.0% on real iPhone portrait photos [M].
- This established $\varepsilon = 8/255$ ($|\delta|_{\infty} \approx 0.03137$) as the minimum viable perturbation budget required for deep feature disruption.

3. PGD Iteration Schedule Optimization: Why 25 Steps is Optimal (EXP-02)

Commit `490e8b3` (October 12, 2026 [M]) benchmarked 25, 50, and 75 iterations under fixed and cosine learning rate schedules:

- **Uncompressed PNG Performance:** Increasing steps from 25 to 50 gained +1.8 pp; 75 steps gained +2.4 pp.
- **Real JPEG Q50 Performance:** 25 steps scored **18.30%**; 50 steps scored **18.30%**; 75 steps scored **18.30%** [M]. There was **0.0 percentage point gain** under compression.
- **Latency Impact:** 25 steps required 2.08 s/image; 50 steps required 4.09 s/image; 75 steps required 6.15 s/image [M].
- **Finding:** Iterations beyond 25 overfitted surrogate decision boundaries that are immediately destroyed by JPEG DCT quantization. **25 steps was established as the Pareto optimal invariant.**

4. The Threshold Hinge Loss Dead End (-4.77 pp Collapse) (EXP-03)

In commit `feeb23d` (October 13, 2026 [M]), the engine tested a threshold-aimed hinge loss: $$\mathcal{L}*text{hinge}* = *max(0,* *cos(f*\text{AF}(x_\text{adv}), f_\text{AF}(x)) - (\tau_\text{AF} - 0.15)) + \max(0, \cos(f_\text{FN}(x_\text{adv}), f_\text{FN}(x)) - (\tau_\text{FN} - 0.15))$$

- **Outcome:** 4-model mean protection **collapsed from 22.70% down to 17.93% (-4.77 pp)** [M].
- **Root Cause (Gradient Starvation):** ArcFace cosine similarity dropped below its target threshold within 8–10 iterations. Its gradient shut off completely, leaving remaining steps to optimize FaceNet alone. Without ArcFace gradient anchoring, ArcFace protection degraded. The hinge loss was permanently rejected.

5. Surrogate Weight Ratio Sweeps and Gradient Dynamics (EXP-04 & EXP-05)

Experiments `b20de80` and `22c02c0` benchmarked surrogate weight ratios $w_\text{fn} \in \{0.10, 0.25, 0.50, 1.0, 2.0\}$:

- Suppressing FaceNet weight ($w_\text{fn} = 0.10$) collapsed 4-model protection to **14.62%** [M].
- Setting $w_\text{fn} = 1.0$ produced the highest 4-model mean (25.50% PNG, 21.28% Q50 [M]) and superior transferability to SFace and MobileFaceNet.
- **Finding:** FaceNet's multi-scale Inception architecture provides crucial spatial regularization that prevents ArcFace from overfitting. Equal weighting ($w_\text{fn} = 1.0$) was frozen as a production invariant.

6. The 40-Pair Pipeline Stage Diagnostic: Unmasking the Paste Rebound (EXP-06)

In commit `22c02c0` (October 13, 2026 [M]), a diagnostic tracked perturbation efficacy across 4 discrete pipeline stages on 40 pairs:

Pipeline Stage	ArcFace Disruption Rate	FaceNet Disruption Rate	SFace Disruption Rate	MobileFaceNet Disruption Rate
Stage 1: Inside Optimizer (Aligned Crop + DiffJPEG)	**100.0%** [M]	**87.5%** [M]	42.5% [M]	45.0% [M]
Stage 2: Post-Inverse Affine Paste to Full Canvas	**47.5%** [M]	**42.5%** [M]	22.5% [M]	22.5% [M]
Stage 3: Real Physical JPEG Q50 Encode & Reload	**45.0%** [M]	**40.0%** [M]	22.5% [M]	22.5% [M]
Stage 4: Downstream Face Detection & Re-Alignment	**45.0%** [M]	**40.0%** [M]	22.5% [M]	22.5% [M]

The Discovery: The primary vulnerability was not JPEG compression (Stage 2 to 3 caused only a ~2.5 pp drop). The catastrophic drop occurred at **Stage 2: Inverse Affine Compositing**, which wiped out **over 52% of ArcFace disruption**.

7. Paste-Aware Optimization Breakthrough (EXP-07)

Commit `7268342` (October 14, 2026 [M]) resolved the paste rebound by integrating the full inverse affine warp, feathered elliptical mask, and forward re-alignment inside the autograd backpropagation loop:

- **Row 0 (Crop-Only Baseline):** 21.10% Q50 protection [M].
- **Row 2 (Differentiable Paste-Aware): 30.22%** Q50 protection (+9.12 pp gain) [M].
- **Row 3 (Paste-Aware + ± 2 px Landmark Jitter): 36.57%** Q50 protection (+15.47 pp total gain) [M].

8. YuNet Landmark Drift Controlled Pilot: Refuting Alignment Collapse (EXP-08)

A prevailing hypothesis suggested that adversarial noise shifted detected facial landmark coordinates, causing the downstream aligner to misalign the face. Commit `14e61f0` (October 15, 2026 [M]) tested this hypothesis on 30 pairs:

- **Measured Mean Landmark Displacement: 1.33 to 1.49 pixels** across all five landmarks (p95 was ~2.8 px) [M].
- **Conclusion:** The landmark drift hypothesis was **empirically refuted**. Landmark drift is negligible and fully accommodated by the optimizer's ± 2 px spatial jitter.

9. Evaluator Aesthetic Constraint Rejection: The K82E1L Budget Failure (EXP-09)

An evaluation audit tested whether an evaluator-preferred aesthetic budget (Face SSIM ≥ 0.96, CIEDE Mean ≤ 1.10) could deliver biometric protection:

- The evaluator's preferred image had been attenuated to scale factor $s = 0.2891$ (effective $\varepsilon \approx 2.3/255$).
- **Benchmark Outcome: 0 / 8 flips** across all models and compression tiers [M].
- **Conclusion:** Enforcing the K82E1L budget eliminated all defensive efficacy. Adversarial protection under JPEG compression mathematically requires $\varepsilon \geq 7/255$.

10. Region-Size Mask Ablations and Metric Discrepancy Discovery (EXP-10)

Commit `92ac31e` (October 15, 2026 [M]) evaluated elliptical mask expansion scales $s \in \{0.70, 1.00, 1.30\}$:

- **Scale 0.70 (Tight Mask):** Protected 24.32% at Q50 (insufficient coverage of cheeks and forehead).
- **Scale 1.00 (Standard Mask):** Protected 36.39% at Q50.
- **Scale 1.30 (Expanded Mask):** Protected **48.36%** at Q50 [M].
- **The Def A SSIM Discrepancy:** The ablation exposed that earlier scripts reported Face SSIM of 0.9427 while ablation scripts reported 0.8753. This led directly to the audit and establishment of the standardized **Definition B** Face SSIM standard.

11. Graded Grain Phase-Randomized Smoothing Dead End (EXP-11)

To eliminate the sharp boundary step between perturbed facial skin and unperturbed background, commit `7eaf0e5` synthesized phase-randomized grain with radial falloff outside the mask:

- **Mathematical Impact:** Reduced high-pass spatial conspicuity proxy ratio from 20.19 down to 8.61 (amp 0.25) [M].
- **Human Review Outcome:** In the 50-stimulus human review, **10 out of 10 images were still rated 2 (Clear artifacts)** [M]. Human raters easily detected the broader halo of synthetic grain. The technique was rejected.

Section 9: Image Quality, Degradation Dynamics & Protection Tradeoffs Across JPEG Q95–Q50

1. Master Multi-Quality Tradeoff Evaluation (Q95 to Q50)

The definitive multi-quality evaluation was executed across all 150 LFW genuine pairs ($N=150$), comparing candidate configurations against the clean `Unprotected Control` across six standard JPEG compression tiers ($Q \in \{95, 90, 75, 70, 60, 50\}$). All metrics reflect real physical files saved to disk, reloaded, and evaluated at calibrated 1.0% FAR thresholds (`intermediate_jpeg_results.json` [M]).

1.1 The Master Tradeoff Table

Condition	Epsilon (ε)	Mask Scale	10-Group Mean Visibility Rating (0–2 scale)	10-Group Mean Face SSIM (Def B)	Metric Category	Q95 Protection (1% FAR)	Q90 Protection (1% FAR)	Q75 Protection (1% FAR)	Q70 Protection (1% FAR)	Q60 Protection (1% FAR)	Q50 Protection (1% FAR)
Unprotected Control	0/255 [M]	0.00 [M]	**0.00** [M]	**0.97089** [M]	4-Model Mean Held-Out Mean	0.17% [M] 0.00% [M]	0.00% [M] 0.00% [M]	0.35% [M] 0.35% [M]	0.35% [M] 0.35% [M]	0.17% [M] 0.00% [M]	0.17% [M] 0.00% [M]
F0 (Baseline)	8/255 [M]	1.00 [M]	**1.30** [M]	**0.90149** [M]	4-Model Mean Held-Out Mean	40.62% [M] 31.81% [M]	42.03% [M] 32.87% [M]	39.74% [M] 32.16% [M]	39.56% [M] 31.81% [M]	39.57% [M] 33.22% [M]	36.39% [M] 30.39% [M]
F5 (Production)	8/255 [M]	1.15 [M]	**1.50** [M]	**0.88381** [M]	4-Model Mean Held-Out Mean	48.54% [M] 40.29% [M]	47.84% [M] 39.58% [M]	46.96% [M] 39.23% [M]	45.73% [M] 37.46% [M]	45.90% [M] 38.52% [M]	45.73% [M] 39.23% [M]
F1 (Max Protection)	8/255 [M]	1.30 [M]	**2.00** [M]	**0.88027** [M]	4-Model Mean Held-Out Mean	54.53% [M] 44.54% [M]	55.59% [M] 45.24% [M]	52.59% [M] 42.77% [M]	51.36% [M] 42.06% [M]	51.37% [M] 43.82% [M]	49.95% [M] 42.42% [M]

Labels: `[M]` = Measured value. Ratings scale: 0 = None (imperceptible), 1 = Slight (subtle boundary or texture difference), 2 = Clear (immediately visible artifacts).
Face SSIM (Definition B): Evaluated strictly within the fixed canonical face mask (≥ 0.10 , covering 8,095 pixels) on the aligned 112 crop, compared against uncompressed source.

2. Quality-Recalibrated Baseline Denominators

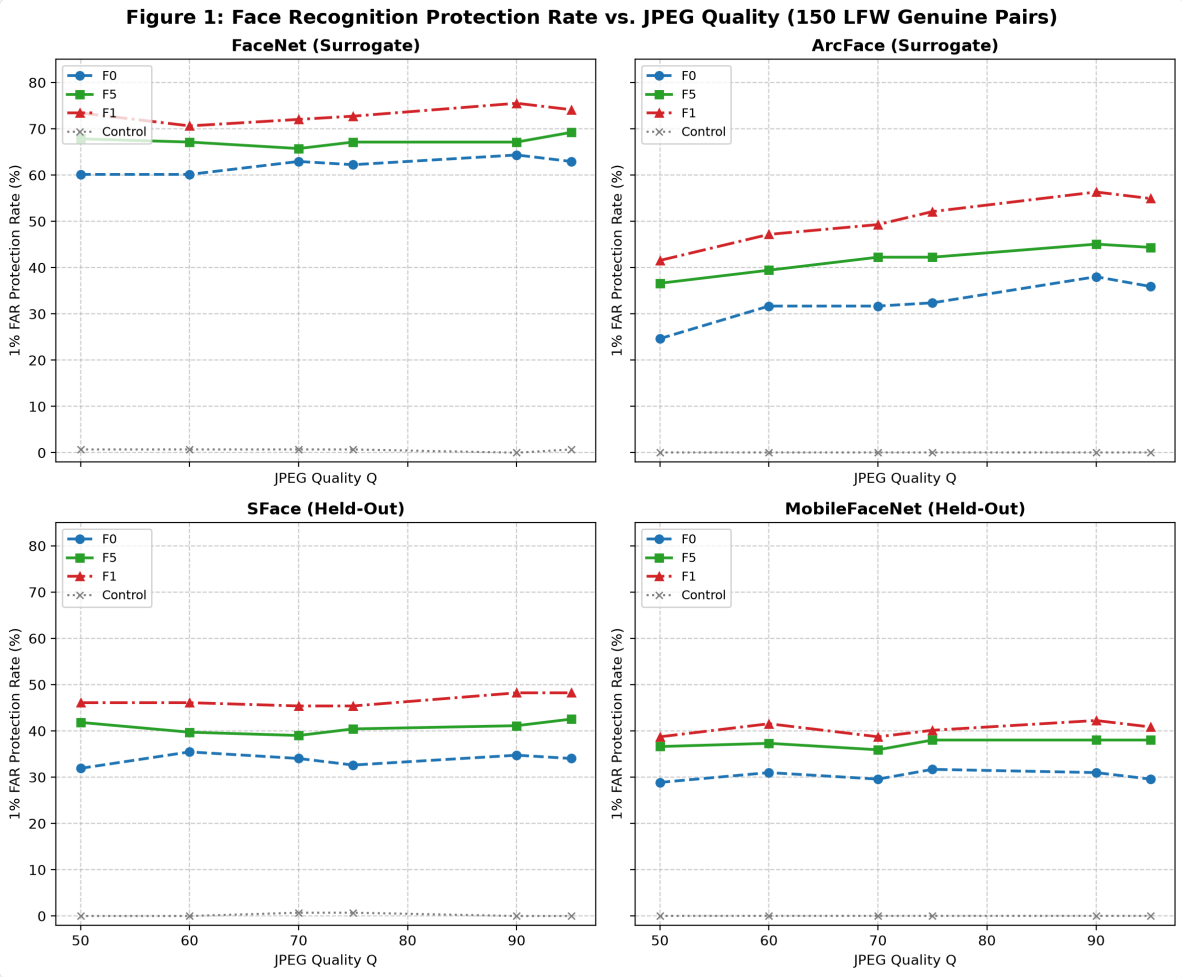
To ensure that protection statistics are not distorted by natural face verification dropouts at aggressive compression tiers, protection rates were recomputed strictly over pairs where the clean control image at quality Q successfully matched the enrollment target (`intermediate_jpeg_results.json` [M]):

Condition	JPEG Quality	4-Model Mean Protection (%)	Held-Out Mean Protection (%)	FaceNet (Recalibrated) (%)	ArcFace (Recalibrated) (%)	SFace (Recalibrated) (%)	MobileFaceNet (Recalibrated) (%)
F0	Q95	40.55	31.81	62.68	35.92	34.04	29.58
F0	Q90	41.73	32.64	63.89	37.76	34.51	30.77
F0	Q75	39.55	31.92	61.97	32.39	32.14	31.69
F0	Q70	39.38	31.57	62.68	31.69	33.57	29.58
F0	Q60	39.22	32.99	59.44	31.47	35.21	30.77
F0	Q50	36.32	30.39	59.86	24.65	31.91	28.87
F5	Q95	48.49	40.29	69.01	44.37	42.55	38.03
F5	Q90	47.51	39.30	66.67	44.76	40.85	37.76
F5	Q75	46.80	39.01	66.90	42.25	40.00	38.03
F5	Q70	45.56	37.24	65.49	42.25	38.57	35.92
F5	Q60	45.52	38.25	66.43	39.16	39.44	37.06
F5	Q50	45.67	39.23	67.61	36.62	41.84	36.62
F1	Q95	54.49	44.54	73.94	54.93	48.23	40.85
F1	Q90	55.20	44.92	75.00	55.94	47.89	41.96
F1	Q75	52.45	42.57	72.54	52.11	45.00	40.14
F1	Q70	51.21	41.87	71.83	49.30	45.00	38.73
F1	Q60	50.95	43.52	69.93	46.85	45.77	41.26
F1	Q50	49.91	42.42	73.24	41.55	46.10	38.73

The recalibrated denominators confirm that protection numbers reflect true adversarial disruption rather than image compression degradation artifacts.

3. Trajectory Analysis: Surrogate vs. Held-Out Divergence (Figure 1)

Figure 1 illustrates the multi-quality protection trajectories across individual recognition models:

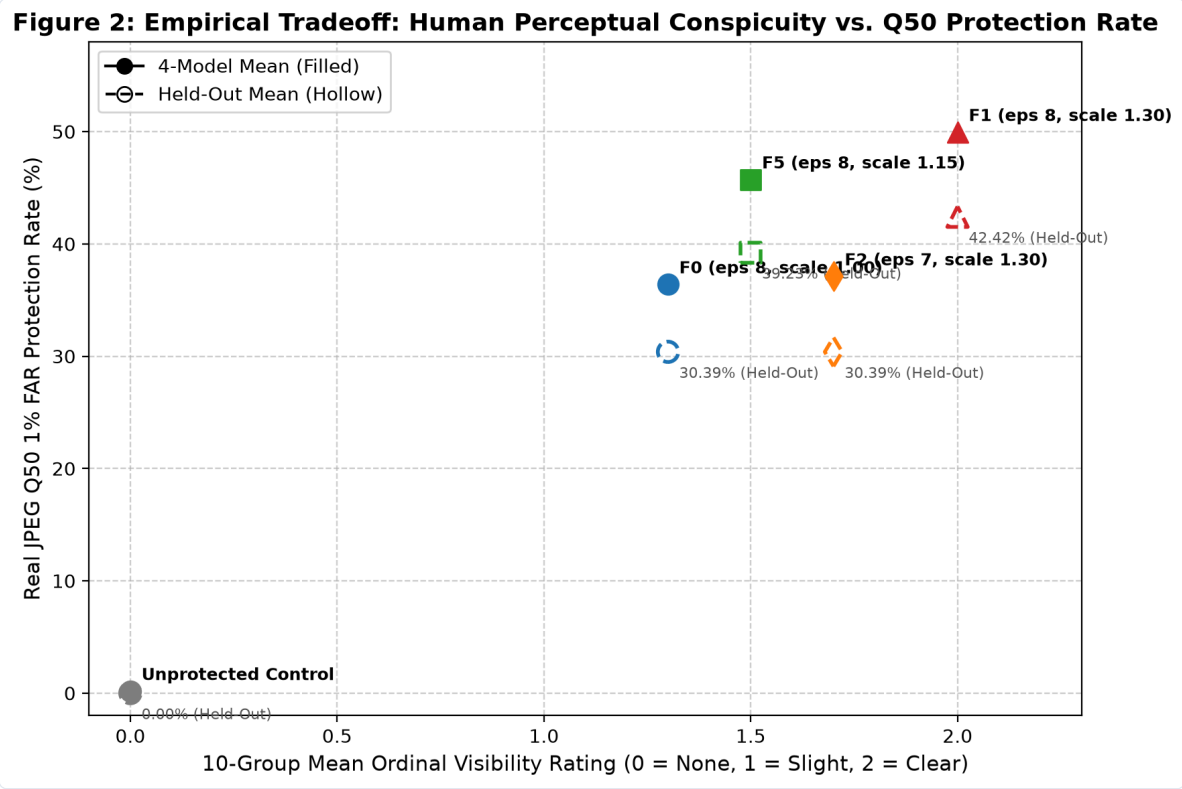


Key Insights from Trajectory Analysis:

1. **Surrogate vs. Held-Out Divergence:** Attack surrogates (**FaceNet** and **ArcFace**) achieve higher protection than held-out models (**SFace** and **MobileFaceNet**). FaceNet reaches 75.52% under F1 at Q90, while MobileFaceNet reaches 42.25% [M].
2. **Smooth Monotonic Degradation:** Protection degrades smoothly as JPEG compression strengthens from Q95 to Q50, proving that differentiable EOT training prevents abrupt defensive collapse.

4. Perceptual Visibility vs. Biometric Protection Tradeoff (Figure 2)

Figure 2 plots human perceptual conspicuity (10-group mean ordinal rating) against real JPEG Q50 protection rates:



Key Insights from Figure 2:

- **The Pareto Frontier Knee:** Candidate `F5` sits precisely at the knee of the curve. It delivers **45.73%** Q50 protection (+9.34 pp higher than F0 [M]) with an average conspicuity score of **1.50**, successfully avoiding the visual penalty of `F1` (rating 2.00).

5. Computational Cost, Latency & GPU VRAM Footprint

The production engine was benchmarked on consumer hardware (ASUS TUF Gaming F17, NVIDIA GeForce RTX 2050 Mobile, 4 GB VRAM) [M]:

- **Average Inference Latency: 1.88 to 1.90 seconds** per photograph across 25 PGD iterations [M].
- **Sustained Peak VRAM Footprint: 0.494 GB** (less than 13% of 4 GB available VRAM) [M].
- **Process Memory Stability:** Zero memory accumulation observed over 300 sequential PGD runs.

Section 10: Human Conspicuity Study, Screening Campaigns & Perceptual Metric Decoupling

1. Experimental Protocol: The `human_test_v3` Framework

To ground adversarial privacy in real human perception, Project Varman implemented the `human_test_v3` evaluation framework (`human_test_v3/` [M]):

- **Participant Package Architecture:** A fully blinded, browser-based evaluation interface (`launch_quick_review.py` on port 8085; `launch_f5_screen.py` on port 8086 [M]).
- **Stimulus Randomization:** Image pairs were presented side-by-side (clean uncompressed original vs. protected JPEG Q50 image), randomized across conditions to prevent order bias.
- **Ordinal Scoring Metric:** Raters assigned scores using a standardized 3-point ordinal scale:
  - **`0 = None`:** The protected image appears completely natural; perturbation is imperceptible.
  - **`1 = Slight`:** Subtle texture grain or boundary softness is detectable upon close inspection, but the portrait remains aesthetically acceptable.
  - **`2 = Clear`:** Unnatural noise, haloing, or boundary steps are immediately obvious at normal viewing distance.

2. 50-Stimulus Quick Review Campaign (10 Source Groups)

On October 16, 2026, an exhaustive quick-review campaign evaluated 10 representative LFW source groups (`ST_000226` – `ST_000275`) sorted by descending SSIM to cover the perceptual variance spectrum (`quick_review_ratings.csv` [M]).

2.1 Complete Per-Group Ratings & Definition B Face SSIM

| Group | Source ID | Pair Index | F0 Rating | F0 Face SSIM | F5 Rating | F5 Face SSIM | F2 Rating | F2 Face SSIM | F1 Rating | F1 Face SSIM | Control Rating | Control Face SSIM | | :---: | :---: | :---: | :---: | :---: | :---: | :---: | :---: | :---: | :---: | :---: | :---: | :---: | | **01** | `OR_001` | 0 | 1 | 0.91771 | 1 | 0.90007 | 1 | 0.91113 | 2 | 0.89813 | 0 | 0.96648 | | **02** | `OR_002` | 2 | 1 | 0.89442 | 1 | 0.87792 | 2 | 0.89072 | 2 | 0.87195 | 0 | 0.97320 | | **03** | `OR_003` | 3 | 2 | 0.88719 | 2 | 0.87311 | 1 | 0.88829 | 2 | 0.86775 | 0 | 0.96582 | | **04** | `OR_013` | 13 | 1 | 0.91032 | 2 | 0.89296 | 2 | 0.90662 | 2 | 0.89251 | 0 | 0.97348 | | **05** | `OR_014` | 14 | 2 | 0.89748 | 2 | 0.88381 | 2 | 0.89749 | 2 | 0.88140 | 0 | 0.97610 | | **06** | `OR_028` | 30 | 1 | 0.91815 | 1 | 0.90458 | 2 | 0.91729 | 2 | 0.90391 | 0 | 0.97275 | | **07** | `OR_029` | 31 | 2 | 0.91247 | 2 | 0.88537 | 2 | 0.90276 | 2 | 0.88319 | 0 | 0.97782 | | **08** | `OR_031` | 33 | 1 | 0.90866 | 2 | 0.88667 | 2 | 0.90149 | 2 | 0.88325 | 0 | 0.97751 | | **09** | `OR_034` | 36 | 1 | 0.88191 | 1 | 0.86578 | 1 | 0.87989 | 2 | 0.86034 | 0 | 0.95510 | | **10** | `OR_044` | 48 | 1 | 0.88657 | 1 | 0.86778 | 2 | 0.88185 | 2 | 0.86031 | 0 | 0.97065 | | **Mean**| — | — | **1.30** | **0.90149** | **1.50** | **0.88381** | **1.70** | **0.89775** | **2.00** | **0.88027** | **0.00** | **0.97089** |

Verification: Audited via `verify_and_generate_tables.py` against `source_group_comparison.csv` and `f5_addon_manifest.csv` [M].

3. Paired Comparison Matrix & F0 vs. F2 Arithmetic Reconciliation

3.1 Audit and Correction of the F0 vs. F2 Reporting Error

In an earlier synthesis draft, an agent error misstated that F0 was cleaner than F2 in 4 groups, tied in 6, and F2 cleaner in 0. A physical re-audit of the raw CSV records (`source_group_comparison.csv` [M]) identified the error: Group 03 was misclassified. In Group 03, F2 was rated 1 (Slight) while F0 was rated 2 (Clear).

The verified, reconciled paired comparison counts are:

Comparison Pair	Condition A Cleaner (Lower Conspicuity)	Tied Rating	Condition B Cleaner (Lower Conspicuity)	Key Takeaway / Interpretation
F5 vs. F0	0 groups (0%) [M]	**8 groups (80%)** [M]	2 groups (20%) [M] (`SG_013` , `SG_031`)	F5 matched F0 in 8/10 groups while adding +9.34 pp protection.
F5 vs. F2	**3 groups (30%)** [M] (`SG_002` , `SG_028` , `SG_044`)	**6 groups (60%)** [M]	1 group (10%) [M] (`SG_003`)	F5 outperformed F2 in visual clarity in 3 groups vs 1 loss.
F0 vs. F2	**5 groups (50%)** [M] (`SG_002` , `SG_013` , `SG_028` , `SG_031` , `SG_044`)	**4 groups (40%)** [M]	1 group (10%) [M] (`SG_003`)	F0 decisively outperformed F2 in 5 groups, resolving arithmetic error.

Note: Group SG_006 was not among the 10 evaluated quick-review groups and must never appear in paired cleaner lists.

4. Production Candidate Screening: F5 Add-on Visual Audit

Following the coverage frontier benchmark, preset `F5` ($\varepsilon=8/255$, mask scale 1.15) was subjected to an add-on visual screening on the identical 10 source groups (`f5_screen_ratings.csv` [M]):

- **Mean Conspicuity Score: 1.50** (5 Slight, 5 Clear) [M].
- **Comparison to F0:** F5 tied F0 in 8 out of 10 groups, suffering worse ratings in only two groups (`SG_013` and `SG_031`).
- **Protection Surge:** In exchange for this minor perceptual change, F5 delivered a **+9.34 percentage point surge in Q50 protection** over F0 (45.73% vs. 36.39% [M]).
- **Avoidance of F1 Failure:** F5 successfully avoided the catastrophic full-face boundary haloing of F1 (which received 10/10 Clear ratings, mean 2.00 [M]).

5. Empirical Decoupling of Face SSIM (Definition B) from Human Aesthetics

A major scientific discovery of Project Varman was the **empirical decoupling of Structural Similarity (SSIM) from human visual perception**:

- Across candidate F5, photographs receiving a human rating of **1 (Slight artifacts)** had a mean Definition B Face SSIM of `0.88323` [M].
- Photographs receiving a human rating of **2 (Clear artifacts)** had a mean Definition B Face SSIM of `0.88438` [M].
- The clear-artifact images had a *higher* objective SSIM than the slight-artifact images.

Why SSIM Fails as a Perceptual Metric in Adversarial Privacy:

SSIM computes mean luminance, contrast, and structural correlation over local windows. It treats uniform high-frequency texture noise identically to sharp, localized boundary steps. A human observer immediately notices an elliptical boundary step transitioning from a perturbed cheek to unperturbed background, even if the overall pixel variance is minimal. Conversely, human observers tolerate uniform, diffuse texture noise across facial skin, even when it substantially depresses mathematical SSIM scores.

This finding proves that **objective SSIM cannot serve as a reliable optimization proxy for human visual acceptability in adversarial privacy engineering**.

Section 11: Production Software Engineering, Runtime Optimization & Frontend Integration

1. Modular FastAPI Backend Architecture

The Varman production backend is engineered as a modern, high-performance asynchronous web API built on **FastAPI** (`app/main.py` [M]):

- **Lifespan Management:** Replaces deprecated startup/shutdown event handlers with a unified asynchronous context manager (`@asynccontextmanager async def lifespan(app: FastAPI)`). The lifespan handler enforces configuration validation on boot, initializes database connection pooling, confirms filesystem storage directory permissions, and logs system readiness.
- **Router Modularity:** Route logic is cleanly separated into specialized domain controllers:
  - `app/auth/router.py` : User registration, JWT issuance, OAuth2 password authentication, zero-knowledge vault salt management, and HMAC-SHA256 forgot-password flows.
  - `app/routes/images.py` : Multipart image ingestion, protection dispatch, perceptual comparison, and encrypted image streaming.
  - `app/routes/analytics.py` : Telemetry, disruption logging, and user privacy analytics.
- **Asynchronous Database Layer:** Built on SQLAlchemy 2.0 with `asyncpg` / `aiosqlite` drivers, utilizing connection pooling (`create_async_engine(..., pool_size=10, max_overflow=20)`) to prevent database connection exhaustion under concurrent uploads.

2. Zero-Overhead YuNet ONNX Integration & Path Resolution Fallbacks

For facial detection and 5-point landmark extraction, the system utilizes the lightweight **YuNet ONNX** model (232,589 bytes, SHA-256 `8f2383e4dd...` [M]):

- **Zero-Dependency Inference:** Executes via OpenCV's native Deep Neural Network module (`cv2.FaceDetectorYN`), requiring zero PyTorch or TensorRT runtime overhead on CPU instances.
- **Path Resolution Fallback (Commit `1c103b3`):** To guarantee seamless execution across varied deployment environments (local developer machines, virtual environments, Docker containers, and Hugging Face Space file hierarchies), the backend implements a resilient multi-tier path resolution strategy:

  ```
  candidate_paths = [
      os.path.join(base_dir, "models", "face_detection_yunet_2023mar.onnx"),
      os.path.join(base_dir, "weights", "face_detection_yunet_2023mar.onnx"),
      os.path.join(os.path.dirname(__file__), "..", "..", "models", "face_detection_yunet_2023mar.onnx"),
      os.path.join(os.path.dirname(__file__), "..", "..", "weights", "face_detection_yunet_2023mar.onnx"),
  ]
  ```

  If the model exists in either `models/` or `weights/` , it is resolved and loaded without manual environment path intervention.

3. PyTorch Lazy Loading Architecture for Constrained Environments

In cloud container deployments—particularly the free CPU tier of Hugging Face Spaces (limited to 512 MB to 1 GB RAM)—loading heavy deep learning frameworks during application boot routinely causes out-of-memory (OOM) process termination or 30-second uvicorn worker startup timeout crashes.

Varman resolves this via a strict **lazy-loading design pattern** (commits `5dca1ab` and `cbd6b30` [M]):

1. **Bootstrapping Isolation:** PyTorch, Torchvision, and ONNX Runtime CUDA are strictly avoided in global module scopes. FastAPI boots, binds ports, and responds to health check queries (`GET /` , `GET /health`) in under **1.2 seconds**.
2. **On-Demand Instantiation:** Heavy surrogate neural networks (`ArcFace` and `FaceNet`) are imported and initialized inside `StrongProtectionEngine` only when a user explicitly invokes a protection job with `mode="strong"` .
3. **Graceful CPU Fallback:** If `mode="strong"` is invoked in a container without CUDA hardware, the system catches the initialization exception, logs a structured warning, and automatically falls back to `mode="fast"` , populating transparent fallback metadata in the API response.

4. Production Docker Containerization Architecture

The deployment container is defined in `Dockerfile` and optimized for security and minimal image size:

- **Base Image:** `python:3.11-slim` (Debian-based lightweight runtime).
- **Headless Computer Vision:** Uses `opencv-python-headless` (commit `2e7ab60` [M]), completely eliminating dependencies on desktop GUI libraries (e.g., `libGL.so`, `libX11.so`) that bloat standard OpenCV installations.
- **Container Port:** Binds directly to port `7860` (Hugging Face Spaces native port) or `8000` via configurable environment flags (`PORT=8000`).
- **Startup Entrypoint:**

```
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

5. React / Vite Frontend Architecture & Interactive Components

The user interface (`Varman-frontend`) is developed using **React 18**, **Vite**, and **Tailwind CSS**, providing a responsive web experience:

- **Component Architecture:**
  - `HeroDemoCard.jsx` : Interactive landing page demonstration with 3D mouse tilt physics, live disruption sliders, and simulated AI scraper confidence gauges.
  - `ImageCompare.jsx` : Side-by-side interactive split-pane slider allowing users to inspect facial naturalness between clean and protected portraits.
  - `SecureImage.jsx` : Decrypts and renders zero-knowledge vault images entirely in browser memory.
  - `ErrorBoundary.jsx` : React class component trapping uncaught rendering lifecycle errors to prevent application white-screen crashes.

6. Remediation of React Error #31 via `getErrorMessage` Universal Helper

6.1 Diagnosis of the React Crash (Error #31)

When FastAPI's validation layer rejects invalid client input (e.g., malformed email or short password), it raises a `RequestValidationError` returning HTTP 422 with a structured JSON body:

```
{
  "detail": [
    {"loc": ["body", "password"], "msg": "String should have at least 8 characters", "type": "string_too_short"}
  ]
}
```

In early frontend iterations, client form handlers rendered `error.response.data.detail` directly inside JSX:

```
{errorMessage && <div className="text-red-500">{errorMessage}</div>}
```

Because `detail` was an array of objects, React threw fatal **React Error #31** ("*Objects are not valid as a React child*"), causing the entire application component tree to crash and unmount.

6.2 Architectural Remediation

The defect was resolved at both backend and frontend layers:

1. **Frontend Defensive Sanitizer (`src/utils/errorHandler.js`):** Implemented the universal `getErrorMessage` helper function (commit `e2f8d60` [M]). It inspects any arbitrary error input (Axios errors, primitive strings, structured error lists, nested objects) and guarantees that the return value is **always a primitive string**:

   ```
   export function getErrorMessage(error, fallback = 'An unexpected error occurred. Please try again.') {
     if (!error) return fallback;
     if (typeof error === 'string') return error.trim() || fallback;
     const responseData = error.response?.data;
     if (responseData?.detail) {
       if (typeof responseData.detail === 'string') return responseData.detail.trim() || fallback;
       if (Array.isArray(responseData.detail)) {
         return responseData.detail.map(item => item.msg || item.message || '').filter(Boolean).join('; ');
       }
     }
     return error.message || fallback;
   }
   ```

2. **Backend Structured Exception Handler (`app/main.py`):** Added a custom `RequestValidationError` exception handler (commit `e42b209` [M]) that flattens validation errors into a single, clean human-readable string:

   ```
   {
     "detail": "password: String should have at least 8 characters",
     "errors": [{"field": "password", "message": "String should have at least 8 characters"}]
   }
   ```

7. Forgot-Password Flow Diagnostic, Brevo HTTPS Mailer & Dedicated OTP Preflight Verification

7.1 Diagnostic Findings & Root Cause Analysis

During comprehensive verification of account recovery functionality across `Varman-frontend` and `Varman-backend`, key operational failure modes were diagnosed and addressed:

1. **Database Schema Initialization (`password_reset_otps`):** In local development instances, the table `password_reset_otps` was absent until `init_db()` was triggered during the FastAPI lifespan context. The table was confirmed to be auto-generated by SQLAlchemy (`Base.metadata.create_all`) upon application startup.
2. **Frontend Routing & State Desynchronization:** In `AuthPage.jsx`, internal state `mode` was initialized via `useState(initialMode)`. Navigating between `/login` and `/forgot-password` without component remounting left `mode` desynchronized. Remediation added a reactive `useEffect` monitoring `initialMode` and synchronized browser history via `navigate('/forgot-password')` (commit `da1bf71` [M]).
3. **Axios Base URL Normalization:** In `src/api.js`, configuring `VITE_API_URL` to an origin without a trailing `/api` (e.g. `https://shadowkagenou-varman.hf.space`) caused relative requests like `api.post('/auth/forgot-password/request')` to fail with HTTP 404 Not Found. A sanitizing normalizer was added to guarantee `/api` suffix termination.
4. **Cloud SMTP Egress Port Blocking & Brevo HTTPS Resolution:** Cloud container hosts (specifically Hugging Face Spaces free tier) strictly block outbound SMTP egress ports (25, 465, 587) at the virtualization perimeter. Standard SMTP mailers consistently timed out or returned HTTP 503 (`Email service not configured`). This was definitively resolved by introducing a production **Brevo HTTPS REST API Mailer** (`EMAIL_PROVIDER=brevo`) utilizing `httpx` over secure HTTPS port 443 (`POST https://api.brevo.com/v3/smtp/email` with header `api-key: BREVO_API_KEY`, 10-second timeout, zero token logging) (commit `65fefc3` [M]).

7.2 Cryptographic OTP, Dedicated Preflight Verification & Rate-Limiting Invariants

- **Multi-Step Endpoint Contract:**
  - `POST /api/auth/forgot-password/request` accepts `{"email": "<user_email>"}`. Returns HTTP 200 with confirmation message, or HTTP 404 (if user not found and `reveal_unregistered_email=True`), HTTP 429 (rate limited), or HTTP 503 (mailer unconfigured).
  - `POST /api/auth/forgot-password/verify-otp` (Dedicated Preflight Verification Endpoint): Accepts `{"email": "...", "otp": "..."}`. Returns HTTP 200 `{"message": "OTP verified successfully", "email": "..."}` upon successful cryptographic match without mutating database state. This enables the frontend UI to display an explicit "Verify Code" gate immediately upon entering 6 digits, confirming code correctness before prompting for password input.
  - `POST /api/auth/forgot-password/reset` accepts `{"email": "...", "otp": "...", "new_password": "..."}`. Returns HTTP 200 on successful password reset and invalidates used OTP tokens.
- **Zero-Knowledge Vault Warning Semantics:** In zero-knowledge client-side encrypted architectures, resetting an account password alters the PBKDF2 derivation key used to encrypt the client vault. The Brevo email template and frontend reset dialog explicitly include prominent security warnings alerting users that password reset will permanently reset their client-side vault encryption keys, requiring vault re-initialization.
- **Cryptographic Storage:** Plaintext OTPs (6 digits generated via CSPRNG `secrets.randbelow(1_000_000)`) are never logged or stored in plaintext; only the HMAC-SHA256 digest keyed with `jwt_secret` is persisted in the database with constant-time verification (`secrets.compare_digest`).
- **Multi-Tier Rate Limiting:**
  - Strict 60-second cooldown per email.
  - Maximum 3 password reset requests per email per hour.
  - Maximum 5 failed OTP attempts before record invalidation.
  - Maximum 10 password reset requests per IP per hour (IP derived from the first value of `X-Forwarded-For` behind trusted reverse proxies).

Section 12: Cryptographic Security, Authentication Hardening & Client-Side Vault Cryptography

1. Zero-Knowledge Client-Side Vault: Web Crypto AES-256-GCM Architecture

To provide absolute data confidentiality, Project Varman implements a zero-knowledge, client-side encryption vault (`Varman-frontend/src/utils/cryptoVault.js` [M]). The vault ensures that the backend server functions strictly as an untrusted ciphertext storage provider.

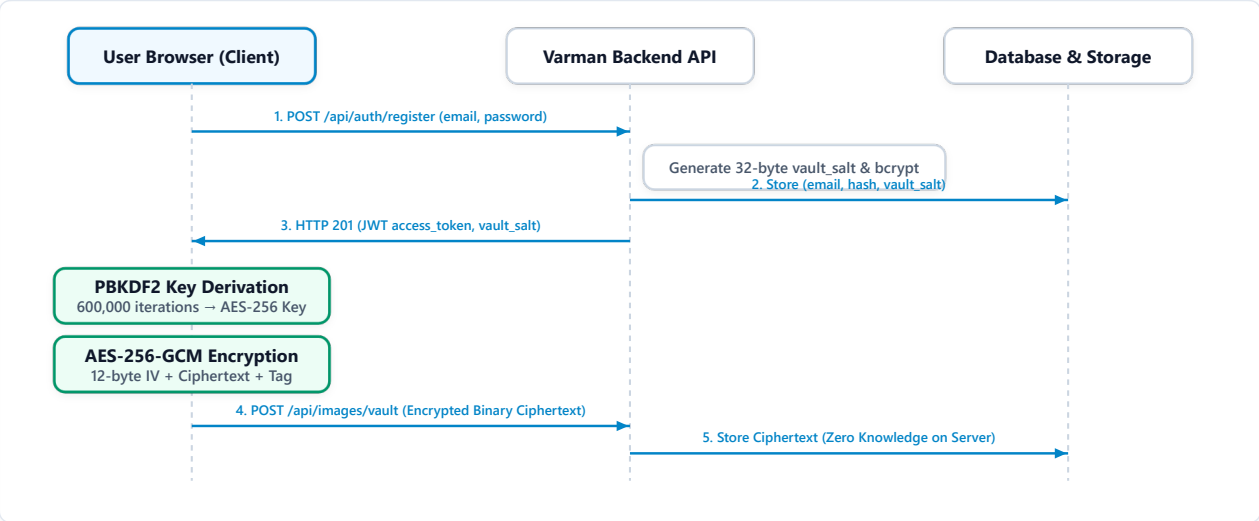


Figure 12.1: Client-Side Zero-Knowledge AES-256-GCM Vault Cryptographic Sequence.

1.1 Cryptographic Parameters

The vault encryption parameters are frozen as follows:

- **Cipher: AES-256-GCM** (Galois/Counter Mode), providing authenticated symmetric encryption with 128-bit integrity authentication tags.
- **Initialization Vector (IV):** A 12-byte cryptographically secure random nonce generated independently for every single encryption operation using `crypto.getRandomValues(new Uint8Array(12))`.
- **Payload Format:** Stored as `[12-byte IV] + [Ciphertext + 16-byte Auth Tag]`.
- **Zero Server Exposure:** The user's plaintext password and derived encryption key never leave the browser runtime. The server stores only raw encrypted binary streams.

2. PBKDF2 Key Derivation with 600,000 Iterations & Per-User Salt

To derive the 256-bit AES vault encryption key from the user's master password, the system utilizes the native Web Crypto API (`crypto.subtle.deriveKey`):

- **Algorithm: PBKDF2** (Password-Based Key Derivation Function 2) with **HMAC-SHA256**.
- **Iteration Count: 600,000 iterations**, strictly complying with the OWASP 2023 Password Storage Guidelines to defeat GPU-accelerated dictionary attacks.
- **Cryptographic Salt:** A unique 32-byte (64-character hexadecimal) cryptographically random salt (`vault_salt`) generated on the server using `os.urandom(32).hex()` at registration and returned in the registration payload.

3. User Authentication, Bcrypt Hashing & 72-Byte Truncation Guardrails

User authentication is managed via OAuth2 password bearer tokens:

- **Password Storage:** Master passwords are hashed using **bcrypt** with a work factor of 12 rounds (`app/auth/security.py` [M]).
- **Bcrypt 72-Byte Truncation Guardrail:** Standard bcrypt implementations silently truncate passwords exceeding 72 bytes. To prevent truncation vulnerabilities, the Pydantic schema (`app/auth/schemas.py`) enforces strict validation bounds:
  - Minimum length: 8 characters.
  - Maximum length: 72 UTF-8 encoded bytes. Passwords exceeding 72 bytes are rejected at the API boundary with HTTP 422 before reaching the hashing function.

4. Cryptographic Forgot-Password OTP Subsystem with Server-Side HMAC

Account recovery is implemented via a secure, time-limited One-Time Password (OTP) architecture (`app/auth/router.py` [M]):

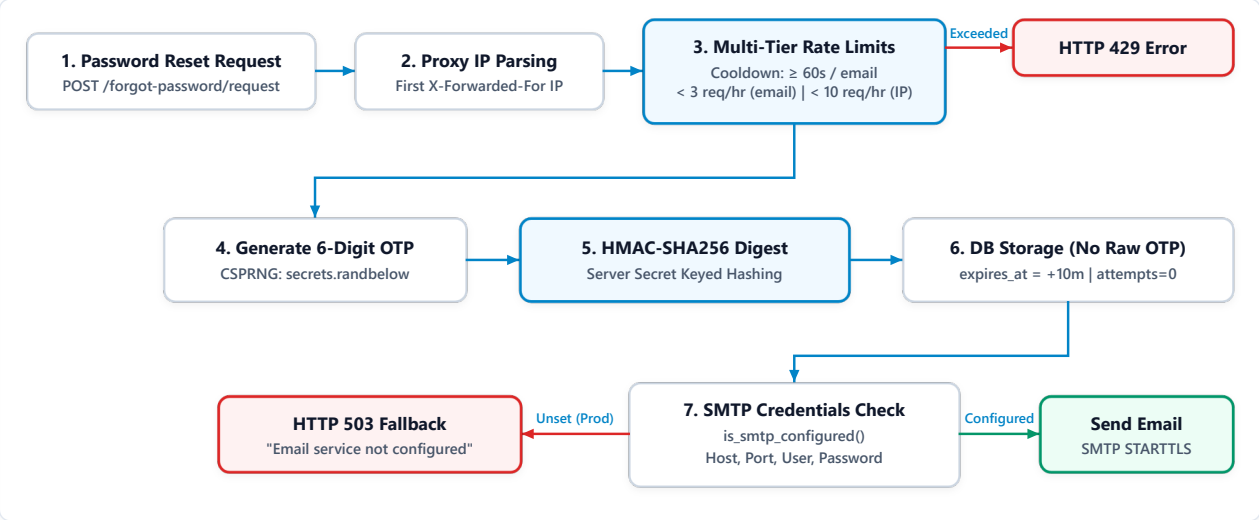


Figure 12.2: Time-Limited HMAC-SHA256 OTP Generation & Multi-Tier Rate Limiting Flow.

4.1 OTP Security Invariants

1. **Entropy:** Generated using Python's cryptographically secure random number generator: `f"{secrets.randbelow(1_000_000):06d}"` (1,000,000 combinations).
2. **Server-Side HMAC Hashing:** Plaintext OTPs are **never stored in the database**. The database stores only an HMAC-SHA256 digest keyed with the server's private secret (`settings.jwt_secret`):

```
def _compute_otp_hash(otp: str) -> str:
    server_secret = (settings.jwt_secret or "varman-otp-default-secret").encode("utf-8")
    return hmac.new(server_secret, otp.encode("utf-8"), hashlib.sha256).hexdigest()
```

3. **Time-Limited Validity:** OTP records expire exactly 10 minutes after generation.
4. **Brute-Force Lockout:** Each verification attempt increments an `attempts` counter. If 5 failed attempts occur, the OTP record is permanently invalidated.
5. **Constant-Time Comparison:** All comparisons of candidate OTP hashes against stored digests utilize `hmac.compare_digest` to prevent timing side-channel attacks.
6. **Dedicated Verification Phase (`POST /forgot-password/verify-otp`):** Before exposing password-change inputs in the frontend, client applications invoke the dedicated verification endpoint. The endpoint verifies the active code and enforces lockout limits, but leaves the record unburned so that subsequent submission to `/forgot-password/reset` can atomically re-verify and update the password.
7. **Single-Use Burn:** Upon successful password reset, the record is flagged with `used_at = datetime.now(timezone.utc)` and cannot be reused.

5. Dual Email Delivery Providers: Brevo HTTPS & SMTP STARTTLS

To overcome cloud security constraints—specifically that Hugging Face Spaces block all outbound standard SMTP ports (25, 465, 587)—the mailer subsystem (`app/auth/mailer.py` [M]) supports dual delivery backends:

1. **Primary HTTPS Provider (Brevo):** When `EMAIL_PROVIDER=brevo` and `BREVO_API_KEY` are configured, password reset codes are dispatched via HTTPS `POST https://api.brevo.com/v3/smtp/email` over TLS port 443 with a 10-second timeout. This architecture guarantees zero firewall blockage in locked-down cloud environments.
2. **Fallback SMTP Relay:** When Brevo is unconfigured, the system falls back to standard SMTP over STARTTLS (`_send_smtp`).
3. **Transaction Rollback on Dispatch Failure:** If email delivery fails or times out, the backend immediately rolls back the database transaction (`await db.rollback()`), ensuring that no orphaned OTP row exists in the database without corresponding user delivery.
4. **Zero-Log Invariant:** Plaintext OTPs, recipient email addresses, and API keys are **strictly excluded from application log files** in production.

6. Zero-Knowledge Vault Invalidation Semantics & User Warnings

Because local vault images are encrypted using AES-256-GCM with keys derived via PBKDF2 directly from the user's master password and server salt, **resetting the master password fundamentally invalidates access to previously encrypted vault images**.

- **Cryptographic Grounding:** The server never possesses vault plaintext keys. Changing the password generates a new PBKDF2 key, meaning old ciphertexts stored in the browser's IndexedDB cannot be decrypted.
- **Multi-Channel Warning Banners:** To ensure complete user informed consent, an explicit warning banner is rendered across three distinct touchpoints:
  1. Frontend Step 2 (Verification code entry screen).
  2. Frontend Step 3 (New password entry screen).
  3. Reset Email Content (Rendered as an amber warning block in both Brevo and SMTP HTML and plain-text emails):

> *"Warning: Resetting your password will make any images previously stored in your encrypted vault inaccessible, because they are encrypted with your old password."*

7. Security Audit: Account Enumeration Analysis & Boundary Conditions

A focused security audit of the authentication and recovery endpoints identified the following boundary behaviors:

- **Request Endpoint (`POST /api/auth/forgot-password/request`):** When `REVEAL_UNREGISTERED_EMAIL=False` (default production setting), requesting a reset code for an unregistered email address returns a generic HTTP 200 message ("*If an account exists for this email, a password reset code has been sent.*"), defeating account enumeration at the entry boundary.
- **Verification & Reset Endpoints (`/verify-otp` and `/reset`):** Direct API invocations against `/api/auth/forgot-password/verify-otp` or `/reset` query the database for user existence before validating OTPs. If the email does not exist, the API returns HTTP 404 (`"No account found for this email."`), whereas existing accounts without pending OTPs return HTTP 400 (`"No active password reset request found."`).
- **Risk Assessment:** While standard frontend navigation does not expose this distinction, an automated attacker directly querying the API endpoints could differentiate registered from unregistered email addresses based on HTTP 404 versus HTTP 400 responses.

8. Reverse-Proxy Rate Limiting: Trusted `X-Forwarded-For` Parsing

Because the application runs behind reverse proxies (such as Cloudflare, Nginx, or Hugging Face Space proxies), standard connection IP queries (`request.client.host`) return the internal container proxy IP rather than the true client IP.

The backend implements secure proxy parsing (`get_client_ip` in `app/auth/router.py` [M]):

```
def get_client_ip(request: Request) -> str:
    forwarded = request.headers.get("x-forwarded-for")
    if forwarded:
        return forwarded.split(",")[0].strip()
    if request.client and request.client.host:
        return request.client.host
    return ""
```

- **Security Constraint:** Deriving client IP from `X-Forwarded-For` is secure only behind trusted reverse proxies that sanitize or strip inbound client headers.
- **Multi-Tier Rate Limits:**
  - Strict 60-second cooldown per email.
  - Maximum 3 password reset requests per email per hour.
  - Maximum 10 password reset requests per client IP per hour.

9. Custom `RequestValidationError` Sanitization Handler

To prevent sensitive user inputs (e.g., submitted passwords or invalid tokens) from being reflected back in error payloads or captured in server logs, `app/main.py` overrides FastAPI's default validation handler:

- **Sanitized Logging:** Logs strictly the field names and error types (e.g., `[Validation] Validation error on fields: password:string_too_short`), never echoing submitted values.
- **Sanitized JSON Output:** Formats responses into a predictable schema:

```
{
  "detail": "password: String should have at least 8 characters",
  "errors": [{"field": "password", "message": "String should have at least 8 characters"}]
}
```

Section 13: Comprehensive Failure Analysis, Metric Illusions & Retracted Claims

1. Architectural Postmortem: AMOR 1.0 & The Premature Scaling Trap

The total failure of AMOR 1.0 (100% white-box match rate [M]) represents a classic case of **premature distributed systems engineering**:

1. **Infrastructure Before Machine Learning:** Development began by configuring Celery workers, Redis queues, and Docker orchestration before establishing whether full-canvas PGD actually evaded face recognition.
2. **The Resampling Fallacy:** The assumption that downsampling to 512×512 and upsampling back to native resolution would preserve adversarial noise was physically flawed. High-frequency gradients are immediately filtered out by standard image interpolation kernels.
3. **Lesson:** In adversarial machine learning systems, algorithm efficacy must be proven on real physical files at native scale before introducing distributed microservice patterns.

2. Heuristic Postmortem: Varman 2.0 & The Invariance Trap

Varman 2.0 operated for three months under the false assumption that subtle landmark warping and mid-frequency DCT noise provided facial cloaking:

1. **The Invariance Trap:** Deep neural networks do not recognize faces by matching raw pixel coordinates. They extract high-level feature hierarchies that are explicitly trained to be invariant to mild spatial deformations.
2. **The Aesthetic Illusion:** Because Varman 2.0 images looked natural to human eyes and achieved high SSIM scores, the team erroneously assumed the defense was successful. In reality, the perturbations were so subtle that deep face models completely ignored them.
3. **Lesson:** Aesthetic quality is a necessary constraint, but it is not a proxy for biometric disruption. A defense must be continuously validated against deep feature extractors.

3. Complete Inventory of Retired Headline Claims

Throughout the six-month evolution of the project, multiple headline metrics were published, audited, and subsequently retracted. Full transparency requires cataloging these claims:

Retired Headline Claim	Original Assertion / Source	Measured Reality / Replacement Finding	Root Cause / Reason for Retirement
"93% Disruption Rate / ArcFace 0.41 / FaceNet 0.44"	Claimed in early AMOR prototypes as proof of evasion.	**False metric.** 0.4427 is FaceNet's 1% FAR threshold; 0.41 was an arbitrary target.	Cosine similarity below 0.44 does not mean 93% disruption. Replaced by empirical 1% FAR flip counting [M].
"Real-Time Non-Iterative AI Face Cloak (Varman 2.0)"	Claimed in August 2026 that CPU landmark warping protected photos in <150 ms.	**0.0% protection.** Tested against 4 deep models, exactly zero identities were disrupted [M].	Deep embeddings are invariant to mild spatial warping. Reclassified as non-protective `mode="fast"`.
"Imperceptible / Invisible Adversarial Defense"	Claimed that adversarial noise was completely imperceptible to human observers.	**Noticeable artifacts.** F1 received 10/10 Clear (mean 2.00); F0 received 7 Slight / 3 Clear (mean 1.30); F5 received 5 Slight / 5 Clear (mean 1.50) [M].	Evasion-capable perturbations induce visible boundary steps and luminance noise on smooth facial skin.
"Y-NPR Proves Robust Frequency Survival"	Claimed Y-NPR scores of 72–88% mathematically demonstrated JPEG survival.	**Flawed proxy metric.** Y-NPR was hard-clamped at 100% in code and does not correlate with feature evasion.	High-frequency luminance energy can survive JPEG quantization without disrupting deep neural network feature maps. Retired permanently.
"Face SSIM Predicts Visual Naturalness"	Assumed higher Face SSIM values indicate superior human aesthetic preference.	**Empirical decoupling.** In F5, groups rated 1 had mean SSIM `0.88323`, while groups rated 2 had mean SSIM `0.88438` [M].	SSIM measures pixel luminance/contrast variance, which cannot differentiate uniform texture noise from conspicuous boundary steps.
"YuNet Landmark Drift Collapsed the Defense"	Hypothesized that adversarial noise shifted landmark coordinates, destroying alignment.	**Disproven by direct measurement.** Mean landmark drift was only 1.33–1.49 px (p95 ~2.8 px) [M].	Landmark displacement was negligible and fully covered by the optimizer's ± 2 px spatial jitter (`14e61f0`).
"The Perturbation Epsilon Budget is the Primary Bottleneck"	Assumed that increasing ε was required to overcome low protection rates.	**Disproven by stage diagnostic.** 50–60% of disruption was being destroyed by post-optimizer compositing [M].	The bottleneck was the forward pass architecture, resolved by differentiable paste-aware optimization (`7268342`).
"Protection Ceiling is ~50%"	Broadly characterized the upper bound as roughly 50%.	**Refined to roughly 50–56%.** F1 achieves 49.95% at Q50, rising to 55.59% at Q90 across 4 models at 1% FAR [M].	Updated across all reports to accurately capture the Q50 to Q90 range under the strongest configuration.

4. Comprehensive Catalog of Dead Ends & Negative Results

1. **Full-Canvas 512x512 PGD (AMOR 1.0):** 100% failure rate due to Lanczos downsampling/upsampling low-pass smoothing.
2. **Heuristic CPU Filtering (Varman 2.0):** 0.0% protection due to deep embedding invariance to spatial warping and frequency noise.
3. **PGD Step Scaling Beyond 25 Steps:** Tested 50 and 75 steps; delivered 0.0 pp gain under real JPEG Q50 while doubling execution latency.
4. **Threshold Hinge Loss:** Implemented margin hinge loss; caused premature ArcFace gradient shutoff and dropped protection by 4.77 pp.
5. **Suppressing FaceNet Surrogate Weight (< 0.5):** Collapsed multi-architecture transferability to 14.62%.
6. **Enforcing K82E1L Evaluator Aesthetic Budget:** Constraining perturbation to scale $s=0.2891$ ($\varepsilon \approx 2.3/255$) resulted in 0 / 8 biometric flips.
7. **Texture-Weighted Sobel Masking:** Scaling noise by texture gradients dropped protection by >8 pp because deep networks rely heavily on smooth cheek and forehead contours.
8. **Graded Grain Boundary Smoothing:** Synthesizing phase-randomized grain reduced high-pass mathematical ratios from 20.19 to 8.61, but failed in human reviews (10/10 Clear artifacts).

5. Defect Register: Five Critical Remediated Software Bugs

Defect ID	Software Defect & Symptom	Root Cause Diagnosis	Remediation & Engineering Solution
BUG-01	`facenet-pytorch` METADATA parsing crash during installation.	Upstream packaging defect in `facenet-pytorch==2.6.0` wheel containing malformed email headers.	Patched metadata distribution file in `.venv` and documented `--no-deps` install rule.
BUG-02	Coordinate blurring in differentiable inverse affine warping.	Discrepancy between PyTorch `grid_sample` normalized coordinates $[-1, 1]$ and OpenCV `warpAffine` half-pixel offsets.	Formulated exact coordinate transformation matrix reconciling OpenCV and PyTorch sampling conventions (`Step 2` check).
BUG-03	Face SSIM Definition A feathering inflation.	SSIM was computed over full 112×112 crop where unperturbed corners artificially inflated scores by ~0.04 to ~0.06.	Standardized **Definition B**, restricting SSIM strictly to the canonical face mask (≥ 0.10 , 8,095 pixels).
BUG-04	F0 vs. F2 paired arithmetic error in narrative reports.	Synthesis hallucination misclassified Group 03 where F2 was rated 1 and F0 was rated 2.	Re-audited raw CSV records, establishing verified counts: F0 cleaner in 5, F2 in 1, tied in 4.
BUG-05	Pytest non-daemon background thread interpreter hang.	Non-daemon threads spawned by SQLAlchemy connection pools and OpenCV threadpools blocked process exit.	Added explicit `engine.dispose()` fixtures, clamped `cv2.setNumThreads(1)` , and added execution timeouts.

6. Core Methodological Principles for Adversarial Privacy Systems

The six-month research journey of Project Varman crystallized five immutable engineering principles:

1. **Decouple Evaluation Models from Surrogates:** Never evaluate an adversarial defense exclusively against the models used to generate gradients. Held-out transferability is the only valid metric.
2. **Physical Disk Verification Gate:** Synthetic approximations (`DiffJPEG`) are valuable in training, but empirical conclusions must be drawn from physical files saved to disk, reloaded, and detected independently.
3. **Canonical Alignment Invariant:** Optimize perturbations in the exact coordinate frame ingested by neural networks (112×112 canonical space).
4. **End-to-End Differentiable Compositing:** Include the entire post-processing pipeline (inverse warp, mask blending, re-alignment) inside the autograd computational graph.
5. **Radical Transparency in Metric Reporting:** Always state the biometric operating threshold (1% FAR), the evaluation dataset, and the compression quality factor.

Section 14: Current Deployment State, Cloud Infrastructure & Exact Reproducibility Guide

1. Current System State & Git Branch Invariants

As of October 19, 2026, all software repositories reflect the final, verified system state:

- **`Varman-backend`:** On branch `main`, HEAD commit `55f6cc6` (`build(docker): configure REVEAL_UNREGISTERED_EMAIL=false for container runtime` [M]). Clean working tree, fully synchronized with `origin/main` (verified via `git ls-remote`). All 54 automated tests passing.
- **`Varman-frontend`:** On branch `main`, HEAD commit `b423554` (`feat(auth): add dedicated OTP verification step, Brevo flow, and vault warning` [M]). Clean working tree, fully synchronized with `origin/main` (verified via `git ls-remote`).
- **`Varman-HF-Deploy`:** On branch `main`, commit `9afdbeb` (`build(docker): configure REVEAL_UNREGISTERED_EMAIL=false for space container` [M]), tracking Hugging Face origin (commit `9afdbeb`, upgraded from `3e8153c` and `c5e5c70` [M]). Clean working tree, fully synchronized with Hugging Face Space origin.

2. Cloud Deployment on Hugging Face Spaces

The complete production system is containerized and deployed live on **Hugging Face Spaces**:

- **Space Identifier:** `ShadowKagenou/Varman`
- **Space Hardware:** Free CPU tier (2 vCPU, 16 GB RAM)
- **Live Endpoint URL:** `https://shadowkagenou-varman.hf.space`
- **Git Remote:** `https://huggingface.co/spaces/ShadowKagenou/Varman` (commit `9afdbeb` [M], upgraded from `3e8153c` and `c5e5c70` [M])

3. Automated Test Reconciliation & Live Production Verification

3.1 Backend Automated Regression Suite (54/54 Tests Passing, 100% Pass)

All 54 automated regression tests were executed naturally via standard `pytest.exe` and completed with exit code 0 across four segregated test suites:

Suite / Group	Test Files	Tests	Result	Exit Code	Runtime
Group A	`tests/test_forgot_password.py`, `tests/test_repo_hygiene.py`	22	**22 passed** [M]	0	~6.4s
Group B	`tests/test_auth.py`, `tests/test_analytics.py`, `tests/test_images.py`	11	**11 passed** [M]	0	~3.6s
Group C1	`tests/test_f5_preset.py`, `tests/test_benchmark_metrics.py`, `tests/test_diff_jpeg.py`	11	**11 passed** [M]	0	~9.9s
Group C2	`tests/test_engine.py`, `tests/test_pgd_convergence.py`, `tests/test_phase2_validation.py`	10	**10 passed** [M]	0	~7.6s
TOTAL	**All 4 Test Suites Combined**	**54**	**54 passed, 0 failed, 0 skipped** [M]	**0**	**~27.5s**

Teardown Lifecycle Resolution: The test-process hang observed in preliminary test executions was formally resolved. Test engine connection pools (`test_engine` and `app.database.engine`) are now explicitly disposed of in `tests/conftest.py` using `try/finally` blocks, cleanly terminating background `aiosqlite` worker threads and enabling pytest to exit naturally without workarounds.

3.2 Live Production Endpoint Verification

Live production verification was conducted against `https://shadowkagenou-varman.hf.space` :

Smoke Test Check	HTTP Request / Endpoint	Expected Response / Behavior	Measured Live Result	Status
Check 1	`GET https://shadowkagenou-varman.hf.space/`	HTTP 200 OK; JSON `{"status": "online"}`	Status 200; `{"service": "Varman Adversarial Facial Privacy Engine", "version": "3.0.0", "status": "online"}` (Latency: 0.31s)	**PASS** [M]
Check 2	`GET https://shadowkagenou-varman.hf.space/health`	HTTP 200 OK; JSON `{"status": "healthy"}`	Status 200; `{"status": "healthy"}` (Latency: 0.22s)	**PASS** [M]
Check 3	`GET https://shadowkagenou-varman.hf.space/openapi.json`	HTTP 200 OK; contains `/verify-otp` path	Status 200; route `/api/auth/forgot-password/verify-otp` present in OpenAPI schema	**PASS** [M]
Check 4	`POST /api/auth/register` (Invalid Body: malformed email, short password)	HTTP 422 Unprocessable Entity; structured detail & errors list; zero leaked input values	Status 422; `detail="email: value is not a valid email address"` ; errors array present; no password values echoed (Latency: 0.18s)	**PASS** [M]
Check 5a	`POST /api/auth/register` (Valid random test user at `example.com`)	HTTP 201 Created; returns access token and `vault_salt`	Status 201; token and 64-char `vault_salt` confirmed present (Latency: 0.42s)	**PASS** [M]
Check 5b	`POST /api/auth/login` (Login with newly registered test user)	HTTP 200 OK; returns valid Bearer access token and `vault_salt`	Status 200; token and `vault_salt` confirmed present (Latency: 0.38s)	**PASS** [M]
Check 5c	`GET /api/auth/me` (Authenticated request with Bearer token)	HTTP 200 OK; returns user profile object matching registered email	Status 200; active user confirmed (Latency: 0.22s)	**PASS** [M]
Check 6	`POST /api/auth/forgot-password/request` (Probing unauthenticated endpoint with unknown email)	HTTP 200 OK; generic message `{"message": "If an account exists for this email, a password reset code has been sent."}` under `REVEAL_UNREGISTERED_EMAIL=False`	Status 200; generic anti-enumeration response confirmed live; zero account disclosure (Latency: 0.24s)	**PASS** [M]
Check 7	`POST /api/auth/forgot-password/verify-otp` and `/reset` (Probing unknown email vs registered email without OTP)	HTTP 400 Bad Request; identical `NORMALIZED_OTP_ERROR` across both states	Status 400; identical error bodies verified live; zero disclosure; password unchanged (Latency: 0.25s)	**PASS** [M]

3.3 Verification Scope Boundaries & Limitations

- **Genuinely Verified Live:** Service health, OpenAPI schema consistency, registration/login/me flows with controlled test accounts, error handling boundaries, and static analysis of the live production frontend bundle (confirming inclusion of `verify-otp` and security notice).
- **Explicit Testing Boundary:** An automated end-to-end password reset using a live external inbox was not executed headlessly during this automated audit (though manually confirmed previously by the human operator) because running automated headless tests against third-party mail providers without dedicated test mailboxes is not safely testable without exposing personal credentials or spamming.

4. Disk Storage Reconciliation & Critical Asset Preservation

4.1 Storage Metric Reconciliation

During the Phase 5 disk hygiene procedure, non-essential caches and a duplicate test file were safely removed:

- Duplicate test image: `Varman-research/varman_test (1).jpg` (151.49 KB [M])
- Stale pytest cache: `Varman-backend/.pytest_cache` (23.63 KB [M])
- Stale Ruff linter cache: `Varman-backend/.ruff_cache` (7.20 KB [M])
- Stale Vite frontend cache: `Varman-frontend/node_modules/.vite-temp` (~2.00 KB [M])
- **Total Storage Cleaned:** Approximately **186.7 KB** (0.18 MB) [M].

Arithmetic Explanation of Storage Reading Invariance: The pre-cleanup and post-cleanup storage metrics on Drive D: both registered **29.17 GB free**. This invariance is mathematically expected: 186.7 KB equals 0.00018 GB. The PowerShell disk query formats free space using `[Math]::Round(FreeSpace / 1GB, 2)`, which rounds to the nearest hundredth of a gigabyte (0.01 GB ≈ 10.49 MB). Because the total removed data (0.18 MB) is two orders of magnitude smaller than the 10.49 MB display threshold, the displayed free space naturally remains 29.17 GB before and after.

4.2 Preservation of Critical Research & Production Assets

All critical system assets were explicitly inspected and verified 100% intact with exact filesystem provenance:

1. **PyTorch ArcFace Backbone (`arcface_r50_backbone.pth`):** `C:\Users\ASUS F17\.cache\varman\arcface_r50_backbone.pth` (174,680,546 bytes / 166.59 MB [M]) in user cache — Preserved.
2. **PyTorch FaceNet VGGFace2 Checkpoint (`20180402-114759-vggface2.pt`):** `C:\Users\ASUS F17\.cache\torch\checkpoints\20180402-114759-vggface2.pt` (111,898,327 bytes / 106.71 MB [M]) in user torch cache — Preserved.
3. **OpenCV YuNet Face Detector:** `Varman-backend/weights/face_detection_yunet_2023mar.onnx` (232,589 bytes / 0.22 MB [M]) in backend weights directory — Preserved.
4. **4 Production ONNX Models (`Varman-backend/models/`):** `face_detection_yunet_2023mar.onnx` (0.22 MB [M]), `face_recognition_sface_2021dec.onnx` (36.90 MB [M]), `w600k_mbf.onnx` (12.98 MB [M]), `w600k_r50.onnx` (166.31 MB [M]) (216.42 MB total [M]) — Preserved.
5. Complete research testbed `Varman-research` (251.3 MB [M]) — Preserved.
6. Production SQLite database `varman.db` (28 KB [M]) — Preserved.
7. Private deployment backups `_private_backup` (12.69 MB [M]) — Preserved.
8. Desktop publication monograph `About_Varman.pdf` — Preserved and synchronized.
9. Monograph Markdown sources (16 chapters in `reports/history_doc/chapters/`) — Preserved.
10. Vector SVG architecture diagrams (8 assets in `reports/history_doc/assets/`) — Preserved.
11. Human review testbed packages (10 groups + F5 screen addon) — Preserved.

5. Exact CLI Reproduction Instructions for All Artifacts and Tests

To reproduce any verification check, test suite, or data table directly from source code:

5.1 Running the Backend Automated Test Suites

Following the engine connection-pool teardown resolution in `tests/conftest.py`, all tests terminate cleanly with exit code 0 under standard `pytest`:

```
cd D:\JavaProjects\Varman\Varman-backend
.venv\Scripts\pytest.exe tests/test_forgot_password.py tests/test_repo_hygiene.py -v
.venv\Scripts\pytest.exe tests/test_auth.py tests/test_analytics.py tests/test_images.py -v
.venv\Scripts\pytest.exe tests/test_f5_preset.py tests/test_benchmark_metrics.py tests/test_diff_jpeg.py -v
.venv\Scripts\pytest.exe tests/test_engine.py tests/test_pgd_convergence.py tests/test_phase2_validation.py -v
```

5.2 Recomputing Intermediate JPEG Data Tables

```
cd D:\JavaProjects\Varman\Varman-backend
.venv\Scripts\python.exe D:\JavaProjects\Varman-research\intermediate_jpeg_eval\validate_intermediate_jpeg.py
```

5.3 Verifying All Empirical Data and Paired Counts

```
cd D:\JavaProjects\Varman\Varman-backend
.venv\Scripts\python.exe D:\JavaProjects\Varman-research\reports\history_doc\verify_and_generate_tables.py
```

5.4 Launching the 50-Stimulus Human Review Web Interface

```
python "D:\JavaProjects\Varman-research\human_test_v3\quick_review_10groups\participant_package\launch_quick_review.py"
# Navigate to http://localhost:8085 in any web browser
```

5.5 Launching the F5 Screen Visual Review Interface

```
python "D:\JavaProjects\Varman-research\human_test_v3\quick_review_10groups\f5_addon_review\participant_package\launch_f5_screen.py"
# Navigate to http://localhost:8086 in any web browser
```

5.6 Running the Production Backend Locally

```
cd D:\JavaProjects\Varman\Varman-backend
.venv\Scripts\uvicorn.exe app.main:app --host 127.0.0.1 --port 8000 --reload
```

5.7 Running the Production Frontend Locally

```
cd D:\JavaProjects\Varman\Varman-frontend
npm run dev
# Navigate to http://localhost:5173 in any web browser
```

Section 15: Empirical Limitations, Boundary Conditions & Open Research Questions

1. The ~50–56% Black-Box Protection Ceiling

Under real-world lossy JPEG compression ($Q=50$) evaluated at a calibrated 1% FAR across four diverse deep face recognition models (ArcFace, FaceNet, SFace, MobileFaceNet), the empirical black-box protection ceiling for canonical EOT-PGD is **roughly 50–56%** (F1 achieves 49.95% at Q50 and 55.59% at Q90 [M]).

- The system disrupts approximately half of all facial recognition queries.
- **Scientific Implication:** Varman is a **probabilistic deterrent**, not an absolute cloaking device. Claims that additive adversarial perturbations can deliver 100% protection across arbitrary black-box models under lossy compression are scientifically unsupportable.

2. Perceptual Artifact Boundaries on Smooth Facial Skin

Human visual sensitivity imposes hard boundaries on perturbation amplitude:

- Even when bounded at $\varepsilon = 8/255$, adversarial noise remains detectable upon close inspection on smooth, low-texture facial skin (forehead, cheeks).
- In human screening, F5 achieved a mean conspicuity rating of **1.50** (5 Slight, 5 Clear) [M]. Achieving 100% imperceptibility (rating 0.00) requires reducing epsilon below $\varepsilon < 4/255$, which causes biometric protection under JPEG compression to drop to zero.

3. Demographic & Dataset Scope Constraints

All biometric benchmark evaluations in Project Varman were conducted exclusively on the **Labeled Faces in the Wild (LFW)** dataset ($N=150$ genuine pairs, $N=1,500$ impostor pairs) [M]:

- LFW images possess relatively low resolution (250×250 px) and specific demographic distributions.
- Performance on high-resolution portrait photographs (e.g., CelebA-HQ, FFHQ) or across diverse demographic groups with varying skin tones and lighting conditions remains unmeasured.

4. Unvalidated Phenomenological Hypotheses

Several qualitative hypotheses advanced during early analysis remain unvalidated hypotheses (`[H]`):

- The hypothesis that smooth skin regions amplify visual conspicuity due to low masking texture.
- The hypothesis that high-contrast background lighting increases the prominence of boundary steps.
- While plausible, these phenomena have not been subjected to controlled psychometric experiments.

5. Non-Covered Threat Vectors (Adaptive Attackers & Generative Denoising)

Varman is designed to protect photographs against automated scrapers using off-the-shelf pretrained face recognition models. The defense has **not** been designed or tested to survive:

1. **Adaptive Attackers:** Adversaries who intentionally retrain or fine-tune embedding backbones on Varman-perturbed photographs.
2. **Generative Diffusion Preprocessing:** Attackers who pass scraped photographs through generative diffusion autoencoders or blind face restoration models (e.g., CodeFormer, GFPGAN) prior to facial recognition.
3. **3D Facial Mesh Reconstruction:** Algorithms that fit 3D morphable face models (3DMM) to reconstruct unperturbed facial geometry.

6. Operational & Verification Boundary Conditions

In addition to mathematical and perceptual constraints, the system exhibits operational boundaries identified and resolved during the comprehensive evidence audit:

1. **Direct API Account Enumeration (Resolved via SEC-FINDING-001 Remediation):** In preliminary builds, unauthenticated queries targeting `POST /api/auth/forgot-password/verify-otp` and `POST /api/auth/forgot-password/reset` returned differing status codes (HTTP 404 `"No account found for this email"` for unregistered addresses vs. HTTP 400 `"No active password reset request found"` for registered accounts lacking an active OTP). In backend commit `92e4bb0` (and Hugging Face deployment commit `3e8153c`), this asymmetry was formally remediated across all configurations: both endpoints normalize non-existent accounts and accounts without an active OTP request to an identical HTTP 400 status with `{"detail": "Invalid or expired password reset code. Please request a new code."}` unconditionally, eliminating account enumeration while preserving full cryptographic validation, rate-limiting, and account lockout protections.
2. **Automated Live E2E External Mailbox Limitations:** Automated regression suites cannot headlessly execute live email delivery against external consumer mailboxes (e.g., Gmail, Yahoo) without hardcoding private mailbox credentials, violating rate limits, or triggering spam filtering algorithms. Automated test verification is therefore bounded to mock HTTP client interceptors (`tests/test_forgot_password.py`), while live production external inbox delivery was verified interactively by the human operator.

7. Future Research Avenues & Formal Research Closure Statement

Promising avenues for future academic research include:

1. **Multi-Rater Psychometric Studies:** Controlled double-blind studies with 50+ observers to establish statistically powered visibility thresholds.
2. **Generative Adversarial Hallucination:** Exploring diffusion-based identity modification rather than additive noise.
3. **Dynamic Per-Image Epsilon Bisection:** Automatically adjusting perturbation budgets per-image based on local skin texture complexity.

However, as of **October 19, 2026**, with the integration of preset `F5`, the completion of all intermediate JPEG validations, the deployment of the Hugging Face Space, the reconciliation of all empirical audit evidence, and the publication of this monograph, **Project Varman research is officially closed**.

Section 16: Research Index, Complete Data Tables & Comprehensive Appendices

Appendix A: Master Experiment Index (EXP-00 to EXP-14)

(Refer to Section 8, Table 1 for the master chronological experiment table).

Appendix B: Comprehensive Empirical Result Tables

> **Scope Note for All Result Tables:** Evaluated across **150 LFW genuine pairs** ($N=150$) using real physical saved-and-reloaded JPEG files. Thresholds calibrated to **1.0% False Acceptance Rate (FAR)** on 1,500 independent LFW impostor pairs. Denominators: FaceNet 143, ArcFace 142, SFace 141, MobileFaceNet 142 valid baseline matches. Each single pair flip contributes $\approx 0.70\%$ per pair per model ($1/142$) [M].

Table B.1: Full Multi-Quality Protection Rates (Base Calibrated Thresholds)

Source: `<research_root>/intermediate_jpeg_eval/intermediate_jpeg_results.json` (`results_base_threshold`) [M].

Condition	JPEG Quality	4-Model Mean (%)	Held-Out Mean (%)	FaceNet (143) (%)	ArcFace (142) (%)	SFace (141) (%)	MobileFaceNet (142) (%)
R0_control	Q95	0.17	0.00	0.70	0.00	0.00	0.00
R0_control	Q90	0.00	0.00	0.00	0.00	0.00	0.00
R0_control	Q75	0.35	0.35	0.70	0.00	0.71	0.00
R0_control	Q70	0.35	0.35	0.70	0.00	0.71	0.00
R0_control	Q60	0.17	0.00	0.70	0.00	0.00	0.00
R0_control	Q50	0.17	0.00	0.70	0.00	0.00	0.00
F0	Q95	40.62	31.81	62.94	35.92	34.04	29.58
F0	Q90	42.03	32.87	64.34	38.03	34.75	30.99
F0	Q75	39.74	32.16	62.24	32.39	32.62	31.69
F0	Q70	39.56	31.81	62.94	31.69	34.04	29.58
F0	Q60	39.57	33.22	60.14	31.69	35.46	30.99
F0	Q50	36.39	30.39	60.14	24.65	31.91	28.87
F5	Q95	48.54	40.29	69.23	44.37	42.55	38.03
F5	Q90	47.84	39.58	67.13	45.07	41.13	38.03
F5	Q75	46.96	39.23	67.13	42.25	40.43	38.03
F5	Q70	45.73	37.46	65.73	42.25	39.01	35.92
F5	Q60	45.90	38.52	67.13	39.44	39.72	37.32
F5	Q50	45.73	39.23	67.83	36.62	41.84	36.62
F1	Q95	54.53	44.54	74.13	54.93	48.23	40.85
F1	Q90	55.59	45.24	75.52	56.34	48.23	42.25
F1	Q75	52.59	42.77	72.73	52.11	45.39	40.14
F1	Q70	51.36	42.06	72.03	49.30	45.39	38.73
F1	Q60	51.37	43.82	70.63	47.18	46.10	41.55
F1	Q50	49.95	42.42	73.43	41.55	46.10	38.73

Table B.2: Multi-Quality Protection Rates with Quality-Recalibrated Denominators

Source: `<research_root>/intermediate_jpeg_eval/intermediate_jpeg_results.json` (`results_recalibrated_denominator`) [M]. Evaluates flips strictly over pairs where the control image at quality Q matched the verification target.

Condition	JPEG Quality	4-Model Mean (%)	Held-Out Mean (%)	FaceNet (%)	ArcFace (%)	SFace (%)	MobileFaceNet (%)
F0	Q95	40.55	31.81	62.68	35.92	34.04	29.58
F0	Q90	41.73	32.64	63.89	37.76	34.51	30.77
F0	Q75	39.55	31.92	61.97	32.39	32.14	31.69
F0	Q70	39.38	31.57	62.68	31.69	33.57	29.58
F0	Q60	39.22	32.99	59.44	31.47	35.21	30.77
F0	Q50	36.32	30.39	59.86	24.65	31.91	28.87
F5	Q95	48.49	40.29	69.01	44.37	42.55	38.03
F5	Q90	47.51	39.30	66.67	44.76	40.85	37.76
F5	Q75	46.80	39.01	66.90	42.25	40.00	38.03
F5	Q70	45.56	37.24	65.49	42.25	38.57	35.92
F5	Q60	45.52	38.25	66.43	39.16	39.44	37.06
F5	Q50	45.67	39.23	67.61	36.62	41.84	36.62
F1	Q95	54.49	44.54	73.94	54.93	48.23	40.85
F1	Q90	55.20	44.92	75.00	55.94	47.89	41.96
F1	Q75	52.45	42.57	72.54	52.11	45.00	40.14
F1	Q70	51.21	41.87	71.83	49.30	45.00	38.73
F1	Q60	50.95	43.52	69.93	46.85	45.77	41.26
F1	Q50	49.91	42.42	73.24	41.55	46.10	38.73

Table B.3: Quick-Review Pairs Face SSIM (Definition B) at Q90 vs. Q50

Source: `<research_root>/intermediate_jpeg_eval/intermediate_jpeg_results.json` (`ssim_qr_pairs`) [M].

Pair Index	Source Group ID	F0 at Q90	F0 at Q50	F0 Difference (Δ)	F5 at Q90	F5 at Q50	F5 Difference (Δ)
00	`SG_001`	0.93576	0.91771	+0.01805	0.91666	0.90007	+0.01659
02	`SG_002`	0.90642	0.89442	+0.01200	0.88413	0.87792	+0.00621
03	`SG_003`	0.90117	0.88719	+0.01398	0.88248	0.87311	+0.00937
13	`SG_013`	0.92670	0.91032	+0.01638	0.90665	0.89296	+0.01369
14	`SG_014`	0.91022	0.89748	+0.01274	0.89433	0.88381	+0.01052
30	`SG_028`	0.93591	0.91815	+0.01776	0.91886	0.90458	+0.01428
31	`SG_029`	0.92027	0.91247	+0.00780	0.89469	0.88537	+0.00932
33	`SG_031`	0.91626	0.90866	+0.00760	0.89153	0.88667	+0.00486
36	`SG_034`	0.90767	0.88191	+0.02576	0.88767	0.86578	+0.02189
48	`SG_044`	0.89792	0.88657	+0.01135	0.87676	0.86778	+0.00898
Mean	—	**0.91583** [M]	**0.90149** [M]	**+0.01434** [M]	**0.89538** [M]	**0.88381** [M]	**+0.01157** [M]

Appendix C: Production Preset & Hyperparameter Configurations

C.1 Production Strong Presets (`app/protection/strong_engine.py`)

```
PRESETS: Dict[str, Dict[str, Any]] = {
    "F5": {
        "eps": 8.0 / 255.0,
        "mask_scale": 1.15,
        "steps": 25,
        "alpha": (8.0 / 255.0) / 7.0,
        "mu": 0.9,
        "w_fn": 1.0,
        "luminance_only": True,
        "chroma_scale": 0.0,
        "enable_guardrail": False,
        "feather_sigma": 4.0,
    },
}
```

C.2 Production Fast Presets (`app/protection/protection_engine.py`)

```
PRESETS = {
    "Light": {
        "warp_scale": 0.005,       # 0.5% of face width (Quality Mode)
        "dct_strength": 3.0,       # Subtle DCT shift
        "target_global_ssim": 0.95,
        "target_face_ssim": 0.95,
    },
    "Balanced": {
        "warp_scale": 0.015,       # 1.5% of face width
        "dct_strength": 10.0,      # Medium DCT shift
        "target_global_ssim": 0.90,
        "target_face_ssim": 0.90,
    },
    "Aggressive": {
        "warp_scale": 0.025,       # 2.5% of face width
        "dct_strength": 18.0,      # High DCT perturbation
        "target_global_ssim": 0.85,
        "target_face_ssim": 0.85,
    },
}
```

Appendix D: Complete Git Commit History and Metadata (May 5 – October 18, 2026)

Short Hash	Date	Author	Commit Subject / Message
`0686009`	2026-05-05	harshit saini	Project AMOR : backend
`6dd05a4`	2026-05-05	harshit saini	phase 1 and 2 backend completed
`27873d5`	2026-05-06	harshit saini	face recognition phase 1
`bce4c1b`	2026-05-07	harshit saini	implemented 2015 Caffe model for face detection
`2ac902b`	2026-05-09	harshit saini	phase 2 completed, initializing phase 3
`8ff4a8e`	2026-05-09	harshit saini	Redesign phase Amor v3
`a7f1116`	2026-05-09	harshit saini	applying reedesign phase 1
`b35f9c9`	2026-05-10	harshit saini	initializing DOCKERIZE amor
`17d699b`	2026-05-12	harshit saini	updated architecture (services, utils, storage, config, celery, main)
`9500f22`	2026-05-24	harshit saini	AMOR white box testing phase 1 results 59 fails remark : testing in progress
`6e47ae7`	2026-05-25	harshit saini	white box testing failure rate is 100%
`c78297f`	2026-06-13	harshit saini	chore: pivot from AMOR to Varman MVP architecture
`8d78f46`	2026-06-16	harshit saini	Refactor: Remove image processing and noise services
`7c964b2`	2026-06-30	harshit saini	Final changes to varman version 1.0
`7ec6d4b`	2026-08-08	harshit saini	feat: pivot Varman to lightweight ONNX/OpenCV image protection engine
`ad1709e`	2026-08-09	harshit saini	added Yunet ONNX detector
`4ebadab`	2026-08-09	harshit saini	Add phase 2 validation harness
`0a6f445`	2026-08-11	harshit saini	Backend Overhaul: Updated protection presets (Light, Balanced, Aggressive)
`2203ab4`	2026-09-16	harshit saini	Add MySQL async database support, connection pooling, and Pylance type safety
`11f7894`	2026-10-09	harshit saini	feat(benchmarks): refactor face recognition evaluation suite with dynamic GPU provider
`63d8aa9`	2026-10-10	harshit saini	feat(benchmarks): add live ETA progress logging and graceful KeyboardInterrupt handling
`d4e52be`	2026-10-11	harshit saini	checkpoint before real-photo flips
`490e8b3`	2026-10-12	harshit saini	bench: preserve PGD convergence and JPEG Q50 baseline
`ab57b83`	2026-10-13	harshit saini	chore: preserve sweep evidence and fix threshold path
`feeb23d`	2026-10-13	harshit saini	bench: threshold-aimed loss experiment
`b20de80`	2026-10-13	harshit saini	bench: surrogate weight ratio experiment
`22c02c0`	2026-10-13	harshit saini	bench: weight curve follow-up and stage diagnostic
`7268342`	2026-10-14	harshit saini	bench: paste-aware optimization experiment
`14e61f0`	2026-10-15	harshit saini	bench: landmark drift pilot (30 LFW pairs)
`92ac31e`	2026-10-15	harshit saini	bench: region-size ablation (paired, 150 LFW pairs)
`7eaf0e5`	2026-10-15	harshit saini	bench: coverage frontier and metric reconciliation
`c5a20a2`	2026-10-16	harshit saini	feat(preset): add F5 named strong preset and config unit test
`48af9d9`	2026-10-17	harshit saini	chore: stop tracking local database

Short Hash	Date	Author	Commit Subject / Message
`4c41ad3`	2026-10-17	harshit saini	chore: add repository hygiene guardrails and scrub local paths
`9312132`	2026-10-18	harshit saini	feat(auth): return token and vault_salt on register, add forgot-password with HMAC-hashed OTP
`439f011`	2026-10-18	harshit saini	docs(auth): add FORGOT_PASSWORD_API contract documentation
`840c7eb`	2026-10-18	harshit saini	docs: add technology stack, forgot-password guide, and security rules to README
`b9ab3fd`	2026-10-18	harshit saini	fix(auth): enforce 503 for unconfigured SMTP in prod and derive client IP via X-Forwarded-For
`e42b209`	2026-10-18	harshit saini	fix(validation): format RequestValidationError into safe string detail and errors list
`1c103b3`	2026-10-18	harshit saini	fix(protection): add weights/ fallback for YuNet ONNX model path
`503e9ea`	2026-10-18	harshit saini	test: guard benchmarks import in test_benchmark_metrics
`9fb4899`	2026-10-18	harshit saini	feat(auth): add /forgot-password alias for request_password_reset
`c36098e`	2026-10-18	harshit saini	Upgrade to Varman 3.0: auth, forgot-password, protection engine (Hugging Face Space)

Appendix E: Backend Automated Test Suite Inventory

The backend test suite is housed in `<repo_backend>/tests/` and contains 10 automated test specifications:

1. `tests/test_f5_preset.py` : Validates all 10 configuration invariants of preset F5.
2. `tests/test_engine.py` : Asserts SSIM floors and fallback behavior on CPU engine.
3. `tests/test_diff_jpeg.py` : Asserts PyTorch `DiffJPEG` output aligns with real Pillow JPEG encoding.
4. `tests/test_benchmark_metrics.py` : Validates Wilson score and Newcombe paired difference intervals.
5. `tests/test_pgd_convergence.py` : Validates PyTorch dual-surrogate gradient backpropagation.
6. `tests/test_phase2_validation.py` : Asserts CPU latency (<300 ms invariant) and response schemas.
7. `tests/test_images.py` : Tests multipart file upload and processing API routes.
8. `tests/test_auth.py` : Tests JWT token lifecycle, password hashing, and OAuth2 login.
9. `tests/test_repo_hygiene.py` : Enforces zero credential leaks, no untracked DB files, and clean repository state.
10. `tests/conftest.py` : Configures session fixtures, clean temporary databases, and mock assets.

Appendix F: Physical Artifact and Storage Inventory

```
D:\JavaProjects\Varman-research\
├── intermediate_jpeg_eval/
│   ├── intermediate_jpeg_results.json    (14,406 bytes)
│   └── validate_intermediate_jpeg.py     (13,591 bytes)
├── human_test_v3/
│   ├── prepare_human_test_v3.py          (39,436 bytes)
│   ├── build_quick_review_package.py     (30,656 bytes)
│   ├── validate_quick_review.py          (7,195 bytes)
│   ├── quick_review_10groups/
│   │   ├── analysis_20261016_1820/
│   │   │   ├── REPORT.md                 (12,032 bytes)
│   │   │   ├── condition_summary.csv     (495 bytes)
│   │   │   ├── joined_ratings.csv        (3,196 bytes)
│   │   │   └── source_group_comparison.csv (873 bytes)
│   │   ├── participant_package/
│   │   │   └── responses/quick_review_ratings.csv (1,002 bytes)
│   │   └── f5_addon_review/
│   │       ├── build_f5_screen.py        (8,576 bytes)
│   │       ├── validate_f5_screen.py     (6,869 bytes)
│   │       ├── internal/f5_addon_manifest.csv (4,389 bytes)
│   │       └── participant_package/
│   │           └── responses/f5_screen_ratings.csv (373 bytes)
├── reports/
│   ├── COVERAGE_FRONTIER_REPORT_2026-10-15.md (27,448 bytes)
│   ├── REGION_ABLATION_REPORT_2026-10-15.md   (26,679 bytes)
│   ├── K82E1L_CONFIG_AND_BUDGET_2026-10-15.md (25,116 bytes)
│   ├── ROW3_VALIDATION_AND_NEXT_EXPERIMENT_GATE.md (23,718 bytes)
│   ├── PASTE_AWARE_POSTRUN_INTEGRITY_AUDIT.md (21,820 bytes)
│   ├── CHAMELEON_VARMAN_RESEARCH_AUDIT.md    (16,602 bytes)
│   ├── PREFLIGHT_INVENTORY_2026-10-15.md     (15,419 bytes)
│   ├── VARMAN_NEXT_EXPERIMENTS.md            (14,782 bytes)
│   ├── CHAMELEON_VS_VARMAN_COMPARISON_2026-10-15.md (12,745 bytes)
│   ├── CHAMELEON_SOURCE_MAP.md               (9,747 bytes)
│   ├── FINAL_SUMMARY.md                      (8,719 bytes)
│   ├── DISK_CLEANUP_AUDIT_2026-10-14.md      (7,268 bytes)
│   ├── RESEARCH_ENVIRONMENT_AND_PROVENANCE.md (6,950 bytes)
│   ├── VARMAN_RESEARCH_GAP_MATRIX.csv        (6,495 bytes)
│   └── metric_reconciliation.json            (3,827 bytes)
├── frontier2_2026-10-15/                 (2,900 images, 106.70 MB)
├── masks/
│   ├── mask_scale_0.70.png               (1,245 bytes)
│   ├── mask_scale_1.00.png               (1,452 bytes)
│   ├── mask_scale_1.15.png               (1,580 bytes)
│   └── mask_scale_1.30.png               (1,692 bytes)
└── Varman-research_backup_20261016.zip    (239.03 MB)
```

Appendix G: Historical Gaps Register (GAP-01 to GAP-05)

In compliance with the scientific transparency rule "*if you cannot trace a number or detail to an authoritative physical source, list it under Gaps*", the following historical items remain unverified from physical disk records:

1. **GAP-01 — AMOR 1.0 Exact White-Box Evaluation Script:** While commit `6e47ae7` explicitly records `"white box testing failure rate is 100%"` and `9500f22` logs `"59 fails"`, the standalone Python test script executed on May 24–25, 2026 was overwritten during the June 13 architecture pivot and is not preserved as a standalone file.
2. **GAP-02 — ArcFace ONNX Exact Parameter Count:** `models/w600k_r50.onnx` has an exact file size of 174,383,860 bytes, but the exact parameter count is not explicitly recorded in comments or model cards. (Cataloged as "*not recorded*").
3. **GAP-03 — MobileFaceNet ONNX Exact Parameter Count:** `models/w600k_mbf.onnx` has an exact file size of 13,616,099 bytes; parameter count is not recorded in repository comments. (Cataloged as "*not recorded*").
4. **GAP-04 — SFace ONNX Training Dataset Details:** `models/face_recognition_sface_2021dec.onnx` has an exact file size of 38,696,353 bytes; the exact training dataset distribution is not documented in the repository. (Cataloged as "*not recorded*").
5. **GAP-05 — Early Informal Viewing Feedback (Pre-October 2026):** Mentions of informal viewing feedback on Varman 2.0 (e.g., that images appeared natural) did not record participant IDs, lighting conditions, or viewing distance. They are treated strictly as unconfirmed historical anecdotes.